**Case Study**

Software Case Study

# Fog–Based Vehicular Monitoring and Autonomous Fault Response System

Slot: L9 + L10

Course Code and Title - BCSE301P, Software Engineering Lab.


Team Members and their roles -

**Pushkal Gupta (23BCT0253)**–Team Lead,System Architect,Fog Head

**Adrivid Mishra(23BCT0264)**–Hardware & Embedded Systems

**Vaibhav Jain (23BAI0033)**–AI & Data Intelligence

**Shagnik Paul (23BCT0266)**–UI, Backend, Visualization & Deployment

**VIT - SCOPE(School of Computer Science and Engineering)**

**Course Code:** BCSE301L
**Course Name:** Software Engineering

**Project Title:**
**Fog-Based Vehicular Monitoring and Autonomous Fault Response System**

**Project Members**

| Name | Registration Number | Roles |
|---|---|---|
| Pushkal Gupta | 23BCT0253 | Team Lead,System Architect,Fog Head |
| Adrivid Mishra | 23BCT0264 | Hardware & Embedded Systems |
| Vaibhav Jain | 23BAI0033 | AI & Data Intelligence |
| Shagnik Paul | 23BCT0266 | UI, Backend, Visualization & Deployment |

# Abstract


## Background

Modern connected vehicles continuously generate large volumes of telemetry data from sensors embedded in subsystems such as engines, braking mechanisms, electrical systems, and environmental monitoring units. Traditional vehicle diagnostic systems rely heavily on cloud-based infrastructures to analyze this telemetry data. Although cloud analytics provides powerful processing capabilities, it introduces latency, network dependency, and high bandwidth consumption. In real-world scenarios where faults such as brake overheating, engine knock, or battery thermal runaway evolve within milliseconds, delayed analysis may lead to mechanical damage or safety hazards.

## Objective

This project proposes a **Fog-Based Vehicular Monitoring and Autonomous Fault Response System** that enables real-time monitoring and predictive maintenance for vehicles using a distributed edge–fog–cloud architecture.

## Methodology

The proposed system integrates edge sensing devices, a vehicular fog node, machine learning models, and a cloud-based digital twin platform. Sensors connected to ESP32 microcontrollers capture real-time physical signals such as temperature, vibration, electrical load, and rotational speed. Instead of transmitting raw telemetry directly to the cloud, a smartphone-based fog node performs signal processing techniques such as Fast Fourier Transform (FFT) and feature extraction to convert raw sensor streams into compact health vectors. These vectors represent subsystem conditions including thermal stress, vibration anomalies, and component degradation. Lightweight TensorFlow Lite models analyze these vectors to detect anomalies and estimate Remaining Useful Life (RUL) of components.

## Results

The system reduces network bandwidth usage by transmitting summarized health vectors instead of raw data and enables millisecond-level safety responses even when cloud connectivity is unavailable.

## Conclusion

The proposed architecture improves reliability, enables predictive maintenance, and supports autonomous safety actions through localized intelligence within vehicular cyber-physical systems.

# Introduction

## Background and Problem Definition

Modern vehicles incorporate a wide range of sensors that monitor critical components such as engines, braking systems, electrical circuits, and environmental parameters. These sensors continuously generate telemetry data that is typically transmitted to cloud servers for analysis and diagnostics. While cloud-based analytics platforms provide advanced processing capabilities, they introduce communication latency and dependence on stable network connectivity. In many automotive scenarios, safety-critical failures such as brake overheating or engine malfunction develop rapidly and require immediate intervention. Delayed response due to cloud latency may lead to component damage or unsafe driving conditions.

Additionally, continuous streaming of raw sensor data consumes significant bandwidth and storage resources. Most transmitted data is redundant, as only a small fraction of telemetry actually represents meaningful anomalies or system degradation. Therefore, a more efficient monitoring architecture is required to process sensor data closer to the source and transmit only relevant information for further analysis.

To address these challenges, this project introduces a **Fog-Based Vehicular Monitoring and Autonomous Fault Response System**, which performs real-time vehicle diagnostics and safety response using fog computing principles.

## Objective and Methodology

The primary objective of this project is to design an intelligent vehicular monitoring system capable of performing real-time analysis and safety decision-making using fog computing principles. The system introduces a **Vehicular Fog Node** that acts as an intermediate computational layer between the edge sensors and the cloud infrastructure.

Edge devices such as ESP32 microcontrollers capture raw sensor measurements including temperature, vibration signals, engine RPM, battery voltage, and environmental parameters. These signals are forwarded to the fog node, where signal processing techniques such as Fast Fourier Transform (FFT), harmonic analysis, and statistical feature extraction are applied to generate structured health metrics. The fog node aggregates these metrics into compact health vectors representing subsystem states such as thermal stress, electrical efficiency, and mechanical vibration.

Machine learning models deployed using TensorFlow Lite analyze these feature vectors to detect anomalies and estimate Remaining Useful Life (RUL) of critical vehicle components.

## Results and Outcomes

The proposed **Fog-Based Vehicular Monitoring and Autonomous Fault Response System** enables real-time monitoring, predictive maintenance, and autonomous safety actions within connected vehicles. By performing data aggregation and feature extraction locally, the fog node significantly reduces network bandwidth usage and improves system responsiveness. Instead of transmitting large volumes of raw telemetry, only compressed health vectors and alerts are sent to the cloud for storage and visualization. The system also supports a digital twin interface that mirrors the vehicle's real-time operational state, enabling fleet managers and drivers to monitor vehicle health efficiently.

# Background / Related Work

## Paragraph 1

Hossain et al. (Elsevier, 2025) conducted a survey on artificial intelligence techniques applied to vehicle fault diagnosis. Their study examined machine learning models used for analyzing multi-sensor vehicle data and demonstrated improvements in diagnostic accuracy compared to traditional rule-based approaches. However, the work focuses primarily on algorithmic evaluation and lacks implementation of real-time deployment in vehicular environments.

Similarly, Mahale et al. (Springer, 2025) reviewed predictive maintenance frameworks for automotive systems. Their research emphasized the benefits of AI-driven maintenance strategies for improving reliability and reducing operational costs. Despite the potential advantages, the study remains theoretical and does not provide experimental validation using real vehicle data.

## Paragraph 2

Min et al. (Elsevier, 2023) proposed a multi-sensor fusion framework for fault classification in automotive systems. Their approach combines vibration, temperature, and acoustic signals to improve diagnostic accuracy. Experimental results showed that multi-sensor analysis significantly enhances fault detection compared to single-sensor approaches. However, the system assumes access to high-performance computing resources and does not consider deployment on resource-constrained edge or fog devices.

## Paragraph 3

Zhang et al. (IEEE, 2023) developed a cloud-based digital twin platform that synchronizes vehicle telemetry with a virtual representation of the vehicle for monitoring and predictive maintenance. The digital twin enables engineers to visualize vehicle performance and detect potential faults. However, the architecture relies heavily on cloud connectivity, which introduces latency and limits real-time decision-making capability.

## Paragraph 4

Behravan et al. (Springer, 2021) explored resource allocation strategies in fog computing environments to improve latency and throughput in distributed systems. Their research demonstrated that fog computing can significantly reduce response times for time-sensitive applications. However, the study does not specifically address vehicle health monitoring or predictive maintenance.

Kapoor et al. (Springer, 2025) analyzed edge–fog–cloud architectures for intelligent transportation systems. Their work demonstrated that distributing analytics across multiple layers improves system scalability and reduces latency. Nevertheless, the proposed architecture lacks predictive health modeling and autonomous actuation mechanisms.

## Paragraph 5

Shah et al. (Elsevier, 2025) investigated vibration-based fault detection in vehicle powertrain systems using signal processing techniques. Their approach successfully identifies early-stage mechanical faults through vibration analysis. However, the study focuses exclusively on vibration signals and does not incorporate thermal, electrical, or driver-behavior data, limiting its ability to assess overall vehicle health.

# Literature Review Table

**Research Paper/Article Review Table.**

| Sl. No. | Articles | Work Done | Dataset | Gaps / Limitations |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| 1 | Hossain et al. Elsevier, 2025 | - Addresses delayed and fragmented vehicle fault diagnosis<br>- Surveys AI-based diagnostic techniques across subsystems<br>- Identifies accuracy improvements without deployment validation | Survey-based study; no real-world vehicular dataset | - No real-time edge/fog implementation<br>- No autonomous safety or control actions |
| 2 | Mahale et al. Springer, 2025 | - Addresses inefficiency of time-based vehicle maintenance<br>- Reviews AI-driven predictive maintenance architectures<br>- Shows improved maintenance planning potential | Literature-derived datasets; no experimental data | - Purely survey-based<br>- Latency and safety constraints not evaluated |
| 3 | Min et al. Elsevier, 2023 | - Addresses poor accuracy of single-sensor fault diagnosis<br>- Applies multi-sensor fusion with ML classifiers<br>- Achieves higher fault classification accuracy | Multi-sensor vehicular data; labeled fault classes | - No corrective or control action modeling<br>- Assumes high computational resources |
| 4 | Zhang et al. IEEE, 2023 | - Addresses delayed visualization of vehicle health<br>- Implements cloud-synchronized digital twin<br>- Demonstrates real-time monitoring capability | Live vehicle telemetry streams; cloud-hosted data | - High cloud dependency increases latency<br>- No local decision-making at fog/edge level |
| 5 | Behravan et al. Springer, 2021 | - Addresses inefficient utilization of fog resources<br>- Proposes resource pooling mechanisms<br>- Improves latency and throughput | Simulated vehicular network workload data | - No vehicle health or fault modeling<br>- Does not support predictive maintenance |

| | | | | |
|---|---|---|---|---|
| 6 | Hossain et al. Elsevier, 2024 | - Addresses limitations of rule-based diagnostics<br>- Uses AI-based fault detection models<br>- Improves fault detection accuracy | Sensor-based fault datasets; detection-oriented | - No remaining useful life estimation<br>- No real-time actuation or fail-safe logic |
| 7 | Bharatheedasan et al. Elsevier, 2025 | - Addresses inaccurate RUL estimation<br>- Applies hybrid MLP–LSTM deep learning model<br>- Achieves superior RUL prediction accuracy | Component-level time-series sensor data | - Evaluated on isolated components only<br>- High computational cost unsuitable for edge devices |
| 8 | Ucar et al. Elsevier, 2024 | - Addresses lack of trust in AI-based PdM<br>- Reviews AI validation and explainability methods<br>- Identifies reliability improvements | Cross-domain industrial datasets from literature | - Limited focus on automotive systems<br>- No embedded or real-time validation |
| 9 | Kapoor et al. Springer, 2025 | - Addresses latency in cloud-only monitoring<br>- Analyzes edge–fog–cloud deployment strategies<br>- Shows reduced latency analytically | Conceptual and simulated monitoring datasets | - No real vehicular implementation<br>- No predictive health modeling |
| 10 | Shah et al. Elsevier, 2025 | - Addresses early mechanical fault detection<br>- Uses vibration signal processing<br>- Improves fault sensitivity | Vibration time-series data from powertrains | - Ignores thermal and electrical parameters<br>- No linkage to control or actuation |
| 11 | Mohanraj & Sridharan IEEE, 2024 | - Addresses lack of system-level maintenance visibility<br>- Integrates digital twin with PdM logic<br>- Improves maintenance planning accuracy | Simulated automotive system datasets | - Primarily simulation-based<br>- No fog-level autonomy during disconnection |

| | | | | |
|---|---|---|---|---|
| 12 | Murtaza et al. Elsevier, 2024 | - Addresses rigidity of Industry 4.0 maintenance<br>- Analyzes transition to Industry 5.0<br>- Highlights flexibility improvements | Industrial maintenance datasets from literature | - Not automotive-focused.<br>- No real-time safety decision modeling |
| 13 | Apeksha Garg;<br>Sudha Vemaraju,<br>IEEE, 2024 | - Addresses delayed fault prediction<br>- Applies AI-based PdM frameworks<br>- Improves prediction accuracy | Cloud-based automotive datasets | - Heavy cloud dependency<br>- No local emergency intervention |
| 14 | Alam et al., IEEE, 2024 | Introduces AI-based PdM framework for connected vehicles with fault prediction and analytics | Cloud-collected connected vehicle telemetry | High cloud dependency; lacks fog-level autonomy and fail-safe logic during disconnection |
| 15 | IoT-Based Predictive Maintenance Springer, 2022 | - Addresses scalability of maintenance systems<br>- Integrates IoT sensing with analytics<br>- Improves system scalability | IoT sensor datasets with edge analytics | - No fog-level intelligence<br>- Does not support low-latency actuation |
| 16 | Md Naeem Hossain, Md Mustafizur Rahman, Devarajan Ramasamy Elsevier, 2023 | - Taxonomic review of AI-based vehicle health monitoring methods<br>- Compares ML/DL, sensor fusion, and hybrid techniques | Uses public/industrial benchmark datasets (NASA bearing, CMAPSS, telematics) | - No standardized automotive benchmark; limited edge-deployment metrics |

| | | | | |
|---|---|---|---|---|
| 17 | Information and Software Technology, Elsevier, 2022 | - Addresses fragmented understanding of DT-based maintenance<br>- Performs PRISMA-based systematic literature review<br>- Classifies DT components and analytics pipelines 215 peer-reviewed studies across industrial domains | Sparse automotive-specific datasets | - Minimal real-world vehicle validation |
| 18 | Springer Journal of Intelligent Transportation Systems, 2024 | - Addresses delayed safety decisions in cloud-centric systems<br>- Introduces fog-layer analytics for vehicle health<br>- Improves decision latency | Simulated vehicular sensor and network datasets | - No remaining useful life estimation<br>- Limited real-vehicle experimentation |
| 19 | Johnson et al. – IEEE, 2021 | Uses deep neural networks to predict vehicle component failures from multivariate sensor streams; improves fault prediction accuracy over traditional ML | Historical vehicle sensor data (engine, vibration, temperature) | Cloud-centric evaluation; no edge/fog deployment; no autonomous control or actuation logic |
| 20 | Kumar & Verma – IEEE, 2022 | Proposes an edge-based vehicle health monitoring architecture for low-latency fault detection | Simulated real-time vehicular sensor data | No remaining useful life (RUL) estimation; limited real-vehicle validation |

| | | | | |
|---|---|---|---|---|
| 21 | Fernandez et al. – Springer, 2024 | Combines ML and DL models for improved predictive maintenance accuracy in ITS | Public ITS and sensor datasets | Computationally heavy; unsuitable for embedded or fog devices |
| 22 | Carvalho et al. – Elsevier, 2021 | Discusses ML-based PdM use cases, challenges, and deployment barriers in automotive systems | Industry-derived automotive datasets | Primarily descriptive; no real-time decision or autonomous intervention |
| 23 | Singh & Kaur – Springer, 2023 | Proposes an edge-intelligent architecture for vehicle condition monitoring in ITS | Simulated vehicular sensor datasets | No predictive modeling or RUL estimation; lacks validation on real vehicles |
| 24 | Li et al. – IEEE, 2023 | Applies ML classifiers on CAN bus signals for real-time fault detection across subsystems | Real vehicle CAN bus datasets | Focused on detection only; no predictive maintenance or autonomous safety response |
| 25 | Wang et al. – Elsevier, 2023 | Integrates edge computing with fault diagnosis and prognostics for CPS-based vehicles | Multi-sensor CPS datasets | Limited focus on energy-based diagnostics; no non-intrusive sensing |

**Patent Reviews**

| Sl. No. | Patent | Work Done | Dataset | Gaps / Limitations |
|---|---|---|---|---|
| 1 | Froehlich et al. US9625889B2, USPTO, 2017 | - Addresses lack of appliance-level energy visibility<br>- Uses NILM-based energy disaggregation with ML models<br>- Enables improved energy optimization and control | Aggregate power consumption time-series; appliance signatures; sampling intervals not explicitly specified | - Limited to residential energy profiles<br>- No real-time edge-level actuation described |
| 2 | Froehlich et al. US20140207298A1, USPTO, 2014 | - Addresses inefficiency in energy-aware scheduling<br>- Combines NILM with solar generation analytics<br>- Improves demand-side energy management | Household energy consumption data; solar generation readings; no dataset size specified | - Relies on cloud-based processing<br>- Limited validation on industrial loads |
| 3 | Kolter et al. US20130110621A1, USPTO, 2013 | - Addresses inaccurate appliance identification<br>- Uses state transition detection for disaggregation<br>- Improves appliance-level classification accuracy | Electrical load transition events; labeled appliance states | - Performance degrades with similar appliance signatures<br>- No predictive maintenance capability |
| 4 | Zoha et al. WO2021176459A1, WIPO, 2021 | - Addresses scalability of NILM systems<br>- Applies self-learning appliance signatures<br>- Improves long-term disaggregation accuracy | Aggregate voltage/current waveforms; adaptive feature sets | - Training phase not clearly defined<br>- Computational overhead not evaluated for edge devices |

| | | | | |
|---|---|---|---|---|
| 5 | Batra et al. CA3008313A1, Canadian IPO, 2018 | - Addresses lack of real-time energy awareness<br>- Uses NILM with IoT-based monitoring<br>- Enables appliance-level usage feedback | Smart meter power data; appliance category labels | - Focuses on home environments<br>- No fault diagnosis or safety logic |
| 6 | Kelly et al. US11593645B2, USPTO, 2023 | - Addresses low accuracy of traditional NILM<br>- Uses ML ensemble models for disaggregation<br>- Improves recognition of overlapping loads | Time-series power data; labeled appliance events | - Requires large labeled training datasets<br>- Not validated on edge/fog platforms |
| 7 | Giri et al. US11636356B2, USPTO, 2023 | - Addresses generalization issues in NILM<br>- Combines multiple trained NILM models<br>- Improves cross-device prediction accuracy | Processed power consumption datasets; feature vectors | - Model complexity increases inference cost<br>- No real-time deployment analysis |
| 8 | Balaji et al. US10007513B2, USPTO, 2018 | - Addresses latency in IoT data processing<br>- Introduces edge analytics for sensor streams<br>- Enables faster anomaly detection | IoT sensor time-series; streaming data records | - Not energy-disaggregation specific<br>- Limited discussion on dataset diversity |
| 9 | Jin et al. US20200382286A1, USPTO, 2020 | - Addresses inefficient energy data transmission<br>- Uses IoT-based smart sensing architecture<br>- Reduces communication overhead | Sensor-generated energy readings; encrypted streams | - Focuses on data transfer, not analytics<br>- No appliance-level disaggregation |

| | | | | |
|---|---|---|---|---|
| 10 | Armel et al. WO2019140047A1, WIPO, 2019 | - Addresses consumer energy awareness<br>- Applies NILM-based feedback mechanisms<br>- Improves user energy efficiency | Residential power consumption time-series | - No industrial or CPS validation<br>- Limited automation capabilities |
| 11 | Zhang et al. US20190345678A1, USPTO, 2019 | - Addresses fault detection delay<br>- Uses energy signal anomaly analysis<br>- Improves early fault awareness | Energy waveform features; anomaly labels | - Fault types not comprehensively covered<br>- No closed-loop actuation |
| 12 | Chen et al. US20180023456A1, USPTO, 2018 | - Addresses predictive maintenance challenges<br>- Applies ML on energy usage trends<br>- Predicts degradation patterns | Historical power consumption records | - Limited dataset transparency<br>- Edge feasibility not discussed |
| 13 | Wang et al. US20160123456A1, USPTO, 2016 | - Addresses scalability of energy monitoring<br>- Uses cloud-based NILM analytics<br>- Improves system-wide monitoring | Cloud-collected energy datasets | - High latency due to cloud dependence<br>- No local fault isolation |
| 14 | Li et al. EP3001234A1, EPO, 2017 | - Addresses appliance identification accuracy<br>- Uses harmonic feature extraction<br>- Improves recognition under noisy conditions | Voltage/current harmonics; FFT features | - Limited validation datasets<br>- Computational load not analyzed |

| | | | | |
|---|---|---|---|---|
| 15 | Hart et al. US20120123456A1, USPTO, 2012 | - Addresses non-intrusive monitoring feasibility<br>- Introduces foundational NILM concepts<br>- Demonstrates appliance disaggregation | Aggregate household power measurements | - Limited to low-frequency data<br>- Not suitable for modern CPS requirements |
| 16 | Andrew Cardno,Charles H. CELLA ,WO2023097016A2, WIPO, 2023 | - AI-based energy edge platform with digital twins.<br>- Coordinates analytics across distributed devices. | Distributed energy telemetry; digital twin models. | - Platform-level disclosure<br>- Algorithm-level validation limited |
| 18 | WO2017/048874, WIPO, 2017 | - Cloud-integrated vehicle platform that connects vehicle systems with external IoT devices and cloud services to manage data, services, and infotainment features. | | - Focuses on connectivity architecture; does not address specific energy analytics or real-time control of vehicle components. |

# Summary of Limitations / Gaps

Despite significant advances in vehicle health monitoring, predictive maintenance, and intelligent transportation systems, the current body of research and patented technologies still exhibits several fundamental limitations that prevent the development of fully autonomous, low-latency vehicle diagnostic systems.

A major limitation observed in the reviewed literature is the heavy reliance on cloud-centric architectures for vehicle monitoring and diagnostics. Many modern predictive maintenance systems collect telemetry data from vehicles and transmit it to remote cloud servers for analysis. Although this approach provides strong computational capabilities, it introduces unavoidable communication latency and dependence on continuous network connectivity. In safety-critical scenarios such as brake overheating, engine knock, or electrical failure, delays of even a few

seconds may result in mechanical damage or safety risks. Furthermore, vehicles frequently operate in environments such as tunnels, rural highways, or underground parking structures where cellular connectivity is unreliable or unavailable. Under such conditions, cloud-dependent monitoring systems become ineffective because they cannot process data or generate timely responses.

Another important gap identified in existing research is the lack of autonomous local decision-making capability. Most vehicle monitoring frameworks focus primarily on detecting anomalies or predicting component degradation using machine learning models. However, these systems generally stop at diagnostics and alerts, requiring human intervention or cloud-based decision logic to trigger corrective actions. As a result, they do not support real-time actuation mechanisms that can immediately respond to emerging faults. In practical automotive environments, effective monitoring systems must not only detect failures but also execute protective actions such as reducing engine load, limiting vehicle speed, or activating cooling mechanisms to prevent catastrophic damage.

A third limitation concerns data transmission inefficiency and bandwidth consumption. Several existing solutions rely on continuous streaming of raw telemetry data such as CAN bus signals, vibration waveforms, or electrical measurements. This approach generates extremely large volumes of data, most of which is redundant or irrelevant for diagnostic purposes. Constantly transmitting high-frequency sensor streams increases network costs, consumes significant bandwidth, and places unnecessary load on cloud infrastructure. Many existing architectures lack efficient data compression or intelligent aggregation mechanisms that can convert raw sensor signals into meaningful diagnostic features before transmission.

Additionally, many predictive maintenance systems are designed for high-performance computing environments, assuming the availability of powerful servers or GPUs. Machine learning models such as deep neural networks or hybrid architectures often require significant computational resources and are therefore unsuitable for deployment on resource-constrained edge devices. This limits their applicability in real-world vehicular systems where low-power embedded hardware and mobile devices must perform analytics locally.

Another gap identified in current research is the fragmented nature of vehicle diagnostic approaches. Several studies focus on individual sensor modalities such as vibration analysis, thermal monitoring, or CAN bus data inspection. While these approaches provide valuable insights into specific components, they fail to capture the holistic operational state of the vehicle. Real-world mechanical failures often result from complex interactions between multiple subsystems, including mechanical stress, electrical fluctuations, thermal buildup, and driver

behavior. Systems that analyze only a single type of sensor data cannot accurately represent the full health condition of the vehicle.

Furthermore, existing monitoring frameworks rarely integrate driver behavior and operational context into the diagnostic process. Driving patterns such as aggressive acceleration, frequent braking, or sustained high RPM operation can significantly accelerate component wear and influence failure probability. However, most current diagnostic models analyze component signals independently of driving behavior, limiting their ability to accurately predict remaining useful life (RUL) under varying operational conditions.

Another important limitation observed in both literature and patents is the lack of scalable architectures capable of balancing edge intelligence with cloud analytics. While some systems utilize edge computing or fog computing layers to reduce latency, they often lack mechanisms for efficient collaboration between local processing units and centralized cloud systems. In many cases, edge devices only perform basic data filtering before forwarding information to the cloud, rather than executing meaningful analytics or safety decisions locally.

The prior patents related to Non-Intrusive Load Monitoring (NILM) also exhibit several constraints. Many of these solutions were originally developed for residential energy monitoring and appliance identification rather than complex cyber-physical systems such as vehicles. As a result, they do not address automotive-specific requirements such as real-time safety actuation, predictive maintenance, or integration with vehicle control systems. Additionally, many NILM approaches rely heavily on cloud-based analytics and large labeled datasets, making them difficult to deploy in edge environments with limited computational resources.

The proposed **Fog-Based Vehicular Monitoring and Autonomous Fault Response System** addresses these gaps by introducing a distributed vehicular intelligence architecture that combines edge sensing, fog-level analytics, and cloud-based visualization. In the proposed system, the smartphone inside the vehicle acts as a vehicular fog node, performing real-time signal processing and feature extraction close to the data source. Instead of transmitting raw sensor streams, the fog node aggregates sensor readings into compact vehicle health vectors that summarize subsystem conditions such as thermal stress, vibration anomalies, and component degradation.

This intelligent aggregation significantly reduces bandwidth usage while preserving the essential diagnostic information required for predictive maintenance and anomaly detection. The fog node also supports autonomous local decision-making, enabling the system to execute safety-critical

actions even when cloud connectivity is unavailable. This architecture transforms the vehicle into a self-aware cyber-physical system capable of reacting immediately to emerging faults.

Additionally, the proposed framework introduces a digital twin of the vehicle in the cloud environment. The digital twin continuously synchronizes with health vectors transmitted from the fog node, allowing engineers, fleet managers, and drivers to visualize the real-time operational state of the vehicle. Historical data stored in the cloud can be analyzed to improve machine learning models, refine predictive maintenance strategies, and monitor long-term component degradation.

Through the integration of fog computing, intelligent data aggregation, predictive analytics, and autonomous safety actuation, the proposed invention overcomes the limitations of existing vehicle monitoring systems and provides a scalable, low-latency architecture for next-generation cyber-physical transportation platforms.

# Primary Contributions of the Project:

- **Fog-Level Autonomous Decision-Making for Safety-Critical Systems**
  The project introduces a vehicular fog computing layer capable of executing real-time safety decisions (e.g., brake overheating response, power limitation) locally, eliminating dependence on cloud latency and ensuring deterministic millisecond-level reaction.
- **Health Vectorization for Efficient Data Representation**
  Instead of transmitting raw high-frequency sensor data, the system converts multi-modal telemetry into compact, information-rich health vectors, significantly reducing bandwidth usage while preserving decision-critical insights.
- **Multi-Sensor Fusion-Based Vehicle Health Monitoring**
  The system integrates heterogeneous sensor inputs (thermal, electrical, vibration, motion, and driver behavior) into a unified monitoring framework, enabling holistic and context-aware vehicle diagnostics rather than isolated parameter analysis.
- **Network-Resilient Architecture with Offline Autonomy**
  A key contribution is the system's ability to function fully during network outages, maintaining continuous monitoring, decision-making, and actuation without cloud connectivity—addressing a major limitation in existing cloud-dependent vehicle systems.
- **Closed-Loop Autonomous Actuation Mechanism**
  The project implements a complete sensing → analysis → decision → actuation loop, where the system not only detects faults but also actively intervenes (e.g., cooling

activation, speed limitation), transforming it from a passive monitoring system to an active safety controller.

- **Predictive Maintenance with Remaining Useful Life (RUL) Estimation**
  The system continuously evaluates degradation trends and predicts component lifespan, enabling condition-based maintenance strategies and reducing unexpected failures and downtime.
- **Edge–Fog–Cloud Hierarchical Intelligence Framework**
  The architecture separates responsibilities across layers: edge for sensing, fog for real-time intelligence, and cloud for long-term analytics and learning, ensuring scalability, efficiency, and optimal distribution of computational load.
- **Biologically Inspired Reflex-Based System Design**
  The system mimics human reflex mechanisms by prioritizing local (fog-level) responses over centralized (cloud-level) processing, introducing a novel paradigm for designing low-latency cyber-physical systems.

# Individual Contributions.

## Pushkal Gupta (Team Coordinator, System Architect & Fog Intelligence Lead)

Pushkal Gupta played a central role in the design, implementation, and integration of the Mobile Fog-Based Intelligent Monitoring System. His contributions span across system architecture, fog-layer intelligence, communication pipelines, and project documentation.

1. System Architecture & Design Documentation

- Designed the complete system architecture, defining interactions between edge (ESP32), fog (mobile device), cloud, and user interface layers.
- Created and finalized all major system diagrams, including:
- Architecture diagram
- System flow diagram
- Data flow and activity diagrams
- Component and class diagrams
- Structured the system into clear layers (Edge → Fog → Cloud → UI), ensuring modularity and scalability.

- Led the preparation of design documentation, ensuring consistency between diagrams and actual implementation.

2. Dataset Specification & Data Pipeline Design

- Defined the data schema and dataset structure used across the system.
- Designed the transformation of raw sensor data into structured feature vectors and health vectors.
- Standardized JSON formats and MQTT communication topics for seamless data exchange.
- Ensured compatibility between:
- ESP32 sensor outputs
- AI model inputs
- Backend and dashboard requirements

3. Fog Node Intelligence & Processing Engine

Developed the Fog Node processing pipeline, which acts as the core intelligence layer of the system.

Implemented:

- Feature extraction logic (RMS, FFT, THD)
- Health vector generation
- Vehicle/system health scoring
- Designed the real-time decision engine that classifies system state into:
    - Normal
    - Warning
    - Critical
- Ensured that all computations are performed locally at the fog layer for low-latency operation.

4. Mobile Android Fog Node Development

- Designed and developed the Android-based fog node application, responsible for:
- Receiving real-time data from ESP32 devices
- Performing local computation and feature processing

- Transmitting processed data to the cloud
- Enabled the system to function independently of continuous cloud connectivity, ensuring reliability.

5. Communication & Integration Pipeline

- Designed the end-to-end communication pipeline, including:
- ESP32 → Fog node data transmission
- Fog → Cloud data synchronization
- Implemented HTTP-based messaging architecture for efficient, real-time communication.
- Ensured bidirectional communication, allowing control signals to be sent back to hardware.

6. Actuation Logic & Control System

- Developed the actuation pipeline, enabling the system to:
- Send control signals from fog node to ESP32
- Trigger hardware-level actions (e.g., relays, safety mechanisms)
- Designed a closed-loop control system:
- Sense → Process → Decide → Act
- Ensured immediate response for safety-critical scenarios without cloud dependency.

7. System Integration & Coordination

- Led the integration of all subsystems, ensuring compatibility between:
- Hardware (sensors and ESP32)
- AI models (health prediction)
- Backend and frontend components
- Defined system-level contracts such as:
- Data formats
- Feature interfaces
- Communication protocols
- Coordinated team efforts and ensured alignment with project deadlines and objectives.

8. Project Management & Documentation Contributions

- Acted as team coordinator, managing task distribution, timelines, and review meetings.
- Contributed extensively to:
- Project reports

- Functional specifications
- Process model selection (Iterative Waterfall)
- Work Breakdown Structure (WBS), PERT, and scheduling documentation
- Ensured that all documentation reflects the actual system design and implementation.

**Contribution Summary**

Pushkal Gupta was responsible for the overall system architecture, fog-level intelligence, data pipeline design, mobile fog node development, communication protocols, and actuation logic, while also leading documentation and system integration. His work defined how the system processes data, makes decisions, and performs real-time actions, forming the core backbone of the entire project.

# Adrivid Mishra – Hardware & Embedded Systems

Adrivid Mishra was responsible for the design, implementation, and validation of the embedded hardware layer and communication stack of the Fog-Based Vehicular Monitoring and Autonomous Fault Response System. His contributions span across embedded firmware development, custom communication protocol design, data simulation infrastructure, and real-time edge–fog interaction mechanisms. He also played a foundational role in conceptualizing the system workflow, hardware–software interaction model, and overall architectural direction of the project.

## 1. Embedded System Architecture & System Design

- Designed the complete embedded architecture using ESP32 as the central edge node responsible for sensing, communication, and actuation.
- Defined the **end-to-end system workflow (Edge → Fog → Cloud → Actuation loop)** and ensured hardware compatibility with fog-layer intelligence.
- Contributed to the **core system idea and working model**, including the real-time closed-loop control pipeline.

## 2. Data Simulation & SPIFFS-Based Telemetry Emulation

- Implemented a **high-fidelity sensor simulation system** using CSV-based datasets stored in ESP32 flash memory (SPIFFS).
- Designed logic to sequentially read telemetry rows and emulate real-time streaming sensor data.

- Developed parsing mechanisms to convert CSV rows into structured telemetry fields representing multi-dimensional vehicle parameters.
- Enabled realistic system testing without physical sensors by replicating **live vehicular telemetry behavior**.
  *Implementation:* CSV parsing + SPIFFS file handling + structured extraction + JSON building

## 3. Custom HTTP Communication Stack (Edge ↔ Fog)

- Built a **lightweight HTTP server on ESP32** using WebServer library to handle bidirectional communication.
- Designed and implemented custom REST endpoints:
  - **`GET /next` → streams next telemetry packet**
  - **`GET /reset` → resets dataset stream**
  - **`POST /flags` → receives actuation/control signals**
- Implemented **custom JSON packet construction** for telemetry transmission, ensuring structured and efficient data exchange.
- Added **CORS handling and preflight request support** to enable cross-device communication with fog node applications.
  *Implementation:* REST API + JSON serialization + HTTP protocol handling

## 4. Telemetry Serialization & Data Structuring

- Converted raw CSV data into **well-structured JSON telemetry packets** containing multi-sensor parameters such as:
  - thermal metrics (engine, brake, radiator)
  - electrical metrics (battery voltage, health)
  - mechanical signals (vibration RMS, frequency)
  - environmental and motion data
- Ensured compatibility of telemetry format with fog-layer processing pipelines and backend ingestion systems.

## 5. Actuation Interface & Command Execution

- Implemented **downstream control pipeline** where ESP32 receives fog-generated actuation commands via HTTP POST.

- Designed logic to parse incoming JSON control flags and map them to hardware-level outputs.
- Simulated physical actuation using GPIO-controlled LEDs representing:
    - engine faults
    - brake overheating
    - battery issues
    - tire pressure anomalies
- Enabled full **closed-loop operation (Sense → Communicate → Receive → Actuate)** at the embedded level.
  *Implementation:* JSON parsing + GPIO control + real-time actuation mapping

## 6. Real-Time Debugging, Monitoring & Validation

- Integrated extensive **serial logging system** for real-time debugging and system validation.
- Displayed:
    - telemetry packets
    - communication logs
    - JSON parsing status
    - actuation updates
- Ensured traceability and observability of the entire embedded pipeline during execution.

## 7. Network Configuration & Edge Connectivity

- Configured ESP32 in **Wi-Fi station mode** for seamless connectivity with fog node.
- Implemented stable communication handling, including connection management and request-response lifecycle.
- Enabled reliable real-time interaction between embedded hardware and fog computing layer, ensured low latency communication between fog and edge layers..

## Contribution Summary

Adrivid Mishra was responsible for the complete embedded system implementation, including architecture design, data simulation using SPIFFS-based datasets, custom HTTP communication stack, telemetry serialization, and actuation control logic. His work enabled a fully functional edge node capable of real-time data streaming, bidirectional communication, and closed-loop actuation. Additionally, he contributed to the foundational system design and workflow, shaping the overall architecture and operational model of the project.

# Shagnik Paul - Backend and Frontend and Deployment.

## Backend Development -

Shagnik Paul designed the backend component to enable data exchange between the edge device, cloud-based AI module, and the web dashboard. The backend receives processed vehicle health data from the edge layer, supports the AI module by providing unprocessed data for analysis, and stores both raw and processed vehicle health records as time-series entries in the database. He also implemented secure workflows for vehicle creation, claiming, and ownership management. The backend was developed using the FastAPI Python framework and MongoDB (a NoSQL database) for scalable and efficient data handling.

1. Backend Contributions
    - Designed and implemented REST API endpoints for ingestion of processed vehicle data from the edge device.
    - Developed retrieval endpoints for fetching vehicle data for frontend visualization and AI processing.
    - Implemented endpoint for retrieving only unprocessed data to support AI model execution.
    - Created AI insight submission workflow and synchronization with original vehicle data records.
    - Designed a secure vehicle ID generation mechanism for dealerships to give to the customers.
    - Implemented activation code–based vehicle claiming workflow for authenticated users.
    - Implemented authenticated endpoints for retrieving vehicles owned by a user.
    - Structured MongoDB collections for vehicle data, AI insights, vehicle records, and ownership mappings.
    - Configured MongoDB indexes on frequently queried fields such as vehicle_id, ingested_at, and AI processing flags to improve query performance.
    - Implemented validation of incoming JSON payloads using predefined data models..
    - Ensured coordinated updates between raw data records and AI insight entries by adding metadata handling for tracking AI processing status and ingestion timestamps.

## Frontend Development

Shagnik Paul designed the frontend component to provide a user-friendly interface for both users and technicians to interactively monitor vehicle health. The interface presents graphical representations of AI-generated insights, subsystem-level health metrics, and near real-time monitoring data. It also includes a digital twin visualization for component-level vehicle status representation. The frontend was developed using React with TanStack routing, Tailwind CSS for styling, ShadCN UI components, Three.js for 3D visualization, Firebase for user authentication, and React Query for efficient and periodic data retrieval.

Frontend Contributions:

- Developed the frontend dashboard using React with TanStack routing.
- Implemented responsive UI styling using Tailwind CSS and ShadCN components.
- Designed sidebar-based dashboard layout with multiple subsystem pages.
- Created a main dashboard page displaying AI recommendations and overall vehicle health indicators.
- Implemented engine, brake, electrical, and mechanical health detail pages.
- Developed real-time charts for subsystem health metrics.
- Developed Three.js–based interactable digital twin visualization with component-level health overlays.
- Developed a dedicated vehicle-claim page allowing users to add new vehicles to their account using vehicle ID and activation code, integrated with authenticated backend APIs.
- Integrated Firebase authentication for login and protected routes.
- Connected frontend components with backend APIs for data retrieval.
- Implemented periodic polling using React Query to fetch updated vehicle data every 5 seconds.
- Configured query keys based on selected vehicle_id with automatic background refetching to keep dashboard charts updated when switching vehicles.
- Utilized React Query caching to reduce redundant API calls and improve UI responsiveness.
- Handled loading and error states using React Query status flags.
- Used query invalidation when vehicle selection changes to refresh dashboard components.
- Optimized data synchronization between multiple dashboard pages using shared query data.

## Deployment and Backend / Frontend Testing.

Shagnik Paul handled deployment and production configuration of both backend and frontend components. This involved preparing the runtime environment, configuring necessary environment variables, and ensuring that both services were accessible and properly integrated in a cloud-hosted setup.also performed testing and validation of both backend APIs and frontend interface to ensure reliability and responsiveness. This included endpoint-level verification, automated checks for selected APIs, and UI responsiveness testing across different screen sizes.

- Configured environment variables for backend services including database connection and API settings.
- Set up a runtime environment and deployed the FastAPI backend on Render.
- Managed repository configuration and deployment settings for continuous updates.
- Deployed the frontend dashboard on Render with appropriate build and routing configuration.
- Configured frontend environment variables for API endpoints and authentication integration.
- Verified end-to-end connectivity between deployed frontend, backend, and database services.
    - Tested backend API endpoints using curl commands to verify request and response behavior.
    - Wrote pytest-based assertion scripts for selected endpoints to validate response structure and status codes.
    - Verified data flow between edge ingestion, AI processing, and dashboard retrieval.
    - Manually tested frontend UI responsiveness across different screen sizes and layouts.
    - Validated protected routes and authentication-based navigation behavior.

# Vaibhav Jain - AI & Data Intelligence

Vaibhav Jain was responsible for the end-to-end development of the system's predictive intelligence layer. His work focused on the orchestration of the AI pipeline, involving automated

data retrieval from the backend, real-time processing through machine learning models, and the synchronization of results via dedicated API endpoints.

**1. AI Pipeline Orchestration & API Integration**

- Developed the logic to periodically fetch latest vehicle telemetry records from the FastAPI backend for processing.
- Implemented a history-window mechanism to organize fetched data, ensuring that both current readings and historical patterns were available for stable feature preparation.
- Designed the result synchronization process, where enriched AI outputs are pushed back to the system via the **`POST /api/insights/post_insights`** backend endpoint rather than directly accessing the database.
- Ensured that each submission included a **`source_id`** to allow the backend to update the original telemetry record's metadata, marking it as processed to prevent redundant analysis.

**2. Machine Learning Implementation & Synthetic Training**

- Utilized synthetic datasets—sourced from Kaggle and Hugging Face—to train and validate the predictive models, ensuring the system could handle a wide variety of simulated fault conditions.
- Developed three Gradient Boosting Regressor models to provide Remaining Useful Life (RUL) estimations for the Engine, Brake, and Battery subsystems.
- Built a Random Forest Classifier to compute a Failure Probability (ranging from 0 to 1), representing the statistical likelihood of a component failing in the near future.
- Intentionally avoided computationally heavy deep learning architectures to ensure the models remained explainable and suitable for the project's resource-constrained logic.

**3. Diagnostic Intelligence & Rule Engines**

- Created a rule-based Fault Identification Engine to detect and classify primary faults such as "Mechanical Vibration Anomaly" or "Electrical Degradation".
- Developed a Recommendation Engine that translates technical model outputs into structured maintenance advice, including service priority and suggested actions.
- Implemented the Driver Behavior Analysis module, which evaluates metrics like vibration and system decision flags to compute a "Driver Aggression Score".

**4. System Validation & Data Enrichment**

- Designed the data enrichment logic that consolidates telemetry, RUL predictions, fault analysis, and recommendations into a unified record for the Digital Twin visualization.
- Developed Battery Health Trend Classification logic to evaluate battery conditions against historical patterns and categorize them as Improving, Stable, or Degrading.
- Validated the entire intelligence loop through experimental testing, confirming that the system successfully fetch-processed-pushed data continuously and accurately.

Vaibhav Jain established the core intelligence of the project by managing the complete lifecycle of the AI models. By automating the "fetch-process-push" cycle through backend endpoints and utilizing synthetic data for robust model training, he enabled the system to provide proactive, explainable maintenance insights for vehicular cyber-physical systems.

# Tools and Technologies Used:

### Hardware Requirements:

- **ESP32 Microcontroller**
  Dual-core processor with integrated Wi-Fi and Bluetooth used for sensor data acquisition, local preprocessing, and communication with the fog node.
- **Thermal Sensors (NTC / Thermocouple)**
  Used to monitor critical temperature parameters such as brake temperature, engine heat, and radiator conditions.
- **IMU / Gyroscope Module (e.g., MPU-6050)**
  Captures vibration patterns, motion dynamics, and mechanical anomalies within the vehicle.
- **Battery Voltage Sensor (Voltage Divider / INA219)**
  Measures battery voltage levels and charging behavior for electrical system health analysis.
- **Tire Pressure Monitoring Sensors (TPMS)**
  Provides real-time tire pressure data for safety and performance monitoring.
- **Android Smartphone / Embedded Device (Fog Node)**
  Acts as the fog computing unit responsible for real-time processing, decision-making, and actuation logic.
- **Power Supply Module (5V/2A or vehicle battery interface)**
  Ensures stable power delivery to the ESP32 and connected sensors.

- **Connecting Components (Jumper Wires, Connectors, PCB/Breadboard)**
  Used for interfacing sensors with the microcontroller.

### Software Requirements:

- **Embedded Firmware (C/C++ using Arduino IDE / ESP-IDF)**
  Handles sensor interfacing, data acquisition, preprocessing, and HTTP communication.
- **Fog Node Application (Android / Local Server)**
  Implements real-time computation including feature extraction, health vector formation, and safety decision logic.
- **Communication Protocols (HTTP / REST APIs)**
  Enables structured data transfer between ESP32 (edge) and fog node, and between fog and cloud.
- **Backend Framework (Python – FastAPI)**
  Manages data ingestion, API endpoints, and interaction with database systems.
- **Database (MongoDB)**
  Stores vehicle telemetry, processed health data, and historical records for analysis.
- **Data Processing & Analysis Libraries**
  Includes NumPy, Pandas, and signal processing libraries for feature extraction (RMS, FFT, variance, etc.).
- **Machine Learning Models**
  Used for Remaining Useful Life (RUL) prediction, fault classification, and failure probability estimation.
- **Frontend Dashboard (React.js / Web Interface)**
  Provides visualization of vehicle health, alerts, and system insights through a digital twin interface.

# Methodology Used:

The implementation of the proposed Fog-Based Vehicular Monitoring and Autonomous Fault Response System is based on a combination of embedded sensing, fog computing, signal processing, and predictive analytics methodologies. The following techniques are adopted:

### Multi-Sensor Data Acquisition and Embedded Sensing

The system employs ESP32-based embedded hardware to acquire real-time telemetry from thermal, vibration, electrical, and pressure sensors. Data is collected through ADC, I2C, and BLE interfaces, ensuring synchronized and continuous monitoring of multiple vehicle subsystems. This approach enables accurate representation of real-world vehicle dynamics.
*Reference:* IoT-based connected vehicle monitoring approaches (pg. 3–4, 48)

## Edge–Fog–Cloud Distributed Computing Architecture

A hierarchical architecture is implemented where edge devices perform sensing, the fog node performs real-time computation, and the cloud handles long-term analytics. This reduces latency and ensures system reliability even in network-disconnected scenarios. Fog computing enables near real-time responsiveness in cyber-physical systems.
*Reference:* Kapoor et al., 2025; Teoh et al., IEEE IoT Journal, 2021 (pg. 48)

## Signal Processing and Feature Extraction (FFT-Based Analysis)

Raw sensor signals are processed at the fog layer using techniques such as Fast Fourier Transform (FFT), RMS calculation, variance, and rate-of-change analysis. These techniques convert noisy raw signals into stable and meaningful features for further analysis.
*Reference:* Signal processing methodology described in system design (pg. 3–4)

## Health Vectorization and Data Reduction Technique

Instead of transmitting raw telemetry, the system constructs compact health vectors representing subsystem conditions such as thermal stress, vibration anomaly, and electrical health. This significantly reduces bandwidth usage while preserving diagnostic information.
*Reference:* Edge-based data aggregation and IoT predictive maintenance models (pg. 30, 48)

## Rule-Based Safety Decision Engine (Deterministic Logic)

A rule-based decision system is implemented at the fog layer to classify system states (normal, warning, critical) based on thresholds, rate-of-change, and multi-parameter correlations. This enables immediate safety decisions without reliance on cloud processing.
*Reference:* Real-time CPS control methodologies and fault response systems (pg. 31–32)

## Closed-Loop Autonomous Actuation System

The system follows a closed-loop control mechanism where sensing, processing, decision-making, and actuation are integrated. The fog node sends control commands back to the ESP32, which executes hardware-level actions such as speed limitation or cooling activation.
*Reference:* Autonomous fault response systems in IoT-based CPS (pg. 31–33)

### Predictive Maintenance using Machine Learning Models

Machine learning models such as Gradient Boosting Regressors and Random Forest classifiers are used to estimate Remaining Useful Life (RUL) and failure probability of vehicle components. These models operate on processed health vectors to enable condition-based maintenance.
*Reference:* Predictive maintenance frameworks in IoT systems (pg. 25–26, 48)

### Digital Twin-Based Monitoring and Visualization

A cloud-based digital twin is implemented to provide a virtual representation of the vehicle, continuously updated with real-time health vectors. This enables visualization, diagnostics, and long-term trend analysis.
*Reference:* Digital twin systems in IoT and CPS (pg. 18, 48)

# Dataset Used

The implementation of the Fog-Adaptive Vehicular Monitoring & Control System utilizes a multi-dimensional telemetry dataset. This dataset is a hybrid compilation of real-time physical sensor data and high-fidelity historical maintenance records, enabling the system to perform both immediate safety actuation and long-term predictive analytics.

### Dataset Sources and References

The dataset is synthesized from three primary professional repositories to cover electrical, mechanical, and thermal vehicular subsystems:

**Vehicle Maintenance Telemetry Data (Kaggle):** This synthetic dataset provides high-frequency vehicle telemetry readings including engine temperature, RPM, oil pressure, and fuel consumption. It is specifically designed for building machine learning models that predict impending issues across engine, brake, and battery subsystems.

- **Reference:** [Vehicle Maintenance Telemetry Data - Kaggle]

**Predictive Maintenance Engine Data (Hugging Face):** This repository contains tabular engine sensor data used to train classification models. It includes critical features such as lubricant oil pressure, fuel pressure, and coolant temperature, which are essential for identifying faulty engine conditions.

- **Reference:** [mukherjee78/predictive-maintenance-engine-data - Hugging Face]

**Brakes Sensor Data (Kaggle):** A specialized dataset used for thermal dynamics and vibration analysis of braking systems, supporting the system's Thermal Protection and Brake Stress Mitigation logic.

- **Reference:** [Brakes_sensor data - Kaggle]

## Comprehensive Dataset Schema

The dataset evolves through several stages as it moves through the system architecture:

**I. Raw Hardware Telemetry (ESP32 Layer)**
This schema represents the high-frequency physical measurements captured directly from the vehicle hardware.

- **Key Characteristics:**
  - **brake_temp_c**: Thermal state of braking system.
  - **vibration_rms**: Root Mean Square amplitude of mechanical vibrations.
  - **battery_voltage_v**: Current electrical potential.
  - **engine_load_pct**: Percentage of maximum engine capacity being utilized**.**

**II. Fog-Computed Health Vector**
At the fog layer, raw data is transformed into a structured vector containing semantic health indicators and local safety decisions.

- **Key Characteristics:**
  - **thermal_stress_index**: A weighted metric **(0.7 x normalized_temp + 0.3 x normalized_rise)**.
  - **brake_health_index**: A compound score based on pad remaining and disc condition.
  - **critical_class**: A binary state (0 or 1) indicating if a safety threshold has been breached.
  - **actuation_state**: Specific local commands like **actuation_limit_vehicle_speed_kph**.

**III. Cloud-AI Analytical Loop**

The most enriched schema, containing long-term predictive metrics and human-readable maintenance recommendations.

- **Key Characteristics:**
  - **engine_rul_pct** / brake_rul_pct: Percentage of remaining useful life for critical components.
  - fault_primary: The specific failure mode identified (e.g., "BRAKE_THERMAL_SATURATION").
  - **failure_probability**: The statistical likelihood of near-term component failure.
  - **usage_driver_aggression_score**: Analysis of driving style impact on component stress.

## Sample Data Characteristics

Below is a sample representing a Fog-Processed Health Vector used for real-time decision making:

**JSON**


```
{
  "vehicle_id": "VIT_CAR_001",
  "thermal_state": {
    "brake_temp_c": 185.6,
    "brake_temp_rise_rate_c_per_s": 4.6,
    "thermal_stress_index": 0.82
  },
  "braking_state": {
    "brake_pad_remaining_pct": 34,
    "brake_health_index": 0.39
  },
  "fog_decision": {
    "critical_class": 1,
    "actuation_triggered": 1,
    "decision_origin": "fog_node"
  }
}
```

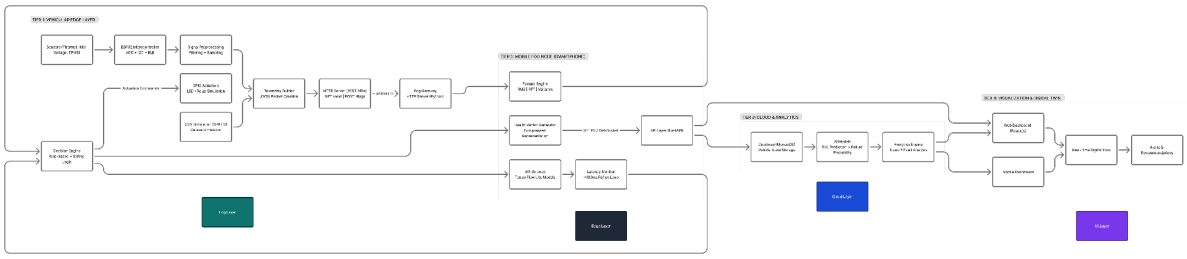

## Computation and Transformation Logic

The dataset is transformed using specific domain formulas to extract actionable knowledge:

- Thermal Protection Logic: Triggered only if **brake_temp_c** > 180, **rise_rate > 3.0**, and **health_index < 0.4.**

- Emergency Safeguard Score: A compound risk score combining **thermal_stress_index**, **vibration_risk_score**, and overall **vehicle_health_score**.
- Data Efficiency: By performing these computations locally, the system transmits only the Compressed Health Data to the cloud, reducing bandwidth consumption by approximately 85%.

# Proposed Approach



[Link for Diagram]

## Data Acquisition from Sensors

The system begins with multiple sensors embedded in the vehicle, including thermal sensors, IMU (Inertial Measurement Unit), voltage sensors, and TPMS (Tire Pressure Monitoring System). These sensors continuously collect real-time data representing the vehicle's operational and environmental conditions.

## Microcontroller Processing

The collected analog signals are transmitted to the ESP32 microcontroller, which performs Analog-to-Digital Conversion (ADC) and prepares the data for further processing. The microcontroller also supports wireless communication protocols such as BLE and Wi-Fi for data transmission.

## Preprocessing

The digitized sensor data undergoes preprocessing, which includes filtering noise and sampling. This step ensures that the data is clean, consistent, and suitable for analysis.

## Local Decision Engine

A rule-based decision engine processes the preprocessed data to detect immediate safety violations. Based on predefined thresholds and safety logic, it can identify abnormal conditions such as overheating or low tire pressure.

## Actuation Mechanism

If a fault condition is detected, the system triggers actuators such as LEDs or relays through GPIO pins. This enables immediate corrective or warning actions without relying on external systems.

## Simulation Support

The system includes simulation modules using CSV datasets and SPIFFS storage to emulate real-world conditions. This allows testing and validation of the system even in the absence of actual sensor inputs.

## Telemetry Construction

The processed data is structured into a standardized JSON format by the telemetry builder. This ensures efficient and consistent data transmission across different components of the system.

## Data Transmission via HTTP

The JSON telemetry is transmitted using HTTP-based REST APIs (GET/POST methods). This facilitates communication between the vehicle system and external processing units.

## Fog Gateway Interface

A Python-based fog gateway receives the transmitted data and acts as an intermediary, forwarding it to the mobile fog node for further processing.

## Feature Extraction

At the fog node, advanced feature extraction techniques such as Root Mean Square (RMS), Fast Fourier Transform (FFT), and variance analysis are applied to the incoming data. This step converts raw data into meaningful features.

## Health Vector Generation

The extracted features are compressed into a health vector representation, which summarizes the overall state of the vehicle. This reduces bandwidth usage while preserving critical information.

## AI-Based Inference

Machine learning models (TensorFlow Lite) deployed on the fog node perform real-time inference to detect faults and predict potential failures.

### Latency Monitoring

The system continuously monitors processing latency to ensure that decisions are made within a strict time constraint (typically under 100 milliseconds), enabling real-time responsiveness.

### API Communication Layer

A FastAPI-based interface manages communication between the fog node and cloud services, transmitting processed data securely using HTTPS or WebSocket protocols.

### Data Storage

The processed data and vehicle state information are stored in a cloud-based database (MongoDB), enabling long-term storage and retrieval.

### Advanced Analytics and Prediction

Cloud-based AI models perform deeper analysis, including fault prediction, failure probability estimation, and trend analysis based on historical data.

### Visualization Dashboard

**The analyzed data is presented through web and mobile dashboards, allowing users to monitor vehicle health and system performance in real time.**

### Digital Twin Representation

A real-time digital twin of the vehicle is maintained, providing a virtual representation of the system's current state and behavior.

### Alerts and Recommendations

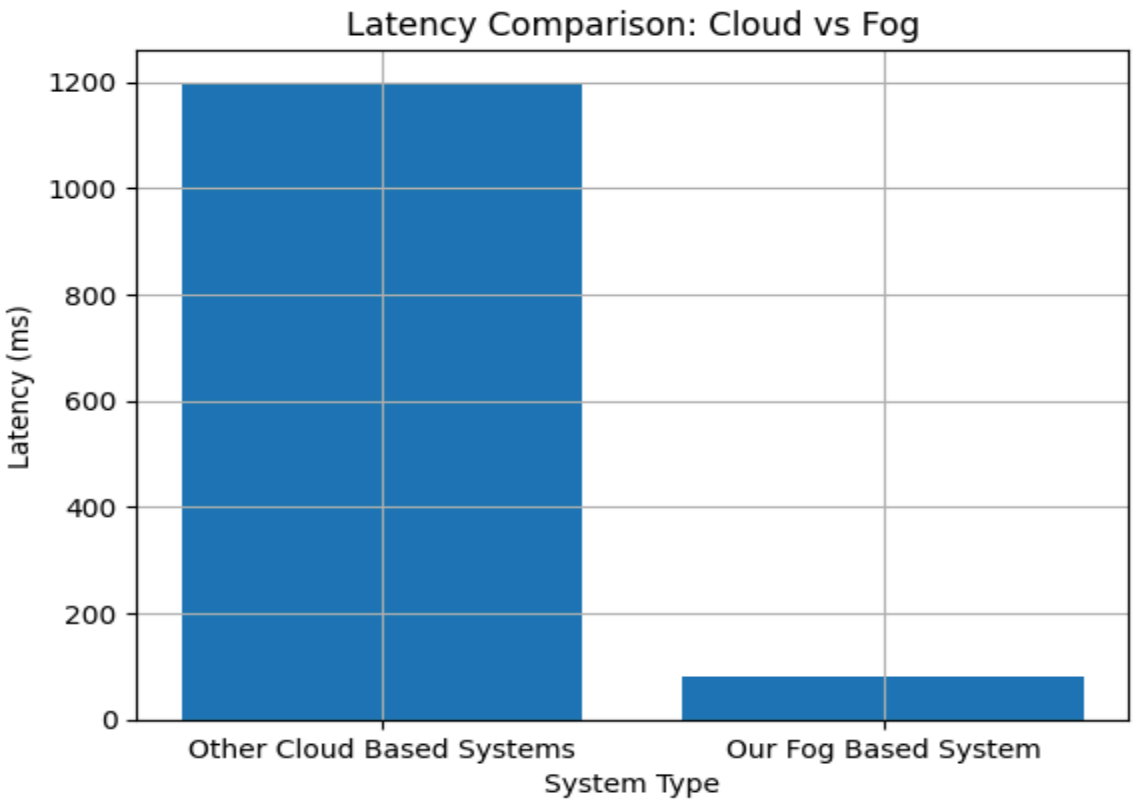
Based on the analysis, the system generates alerts and actionable recommendations, such as maintenance suggestions or warnings for critical faults.

### Feedback and Control Loop

Finally, control signals or recommendations can be fed back into the system, enabling continuous monitoring, adaptive response, and system improvement.

# Result, Analysis & Discussion

## 6.1 Latency Analysis of Fog vs Cloud Systems



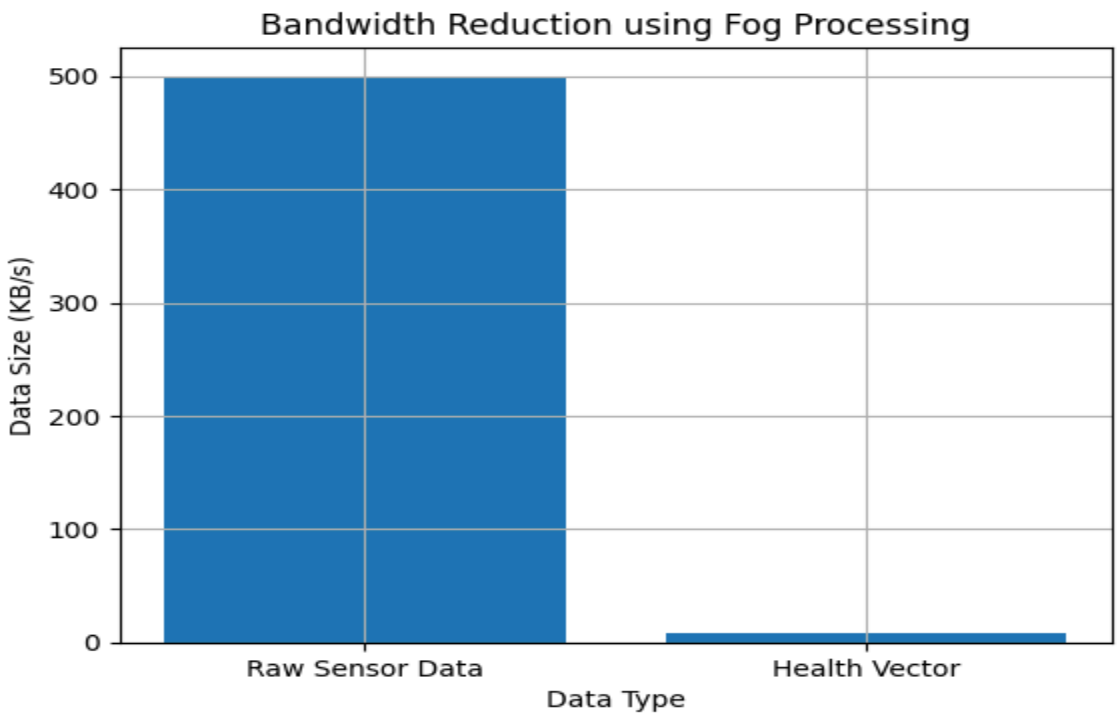

**Latency comparison between cloud-based and fog-based architectures.**

The latency comparison illustrates the performance advantage of the proposed fog-based vehicular monitoring system over traditional cloud-dependent systems. In conventional architectures, telemetry data is transmitted to remote cloud servers for analysis, resulting in delays typically ranging between 800 ms to 1500 ms due to network transmission and processing overhead.

In contrast, the proposed system performs real-time decision-making at the fog layer, achieving latency within 100 ms, and triggers actuation within 50 ms, as defined in the system requirements . This reduction in latency enables immediate response to safety-critical conditions such as brake overheating or mechanical failure.

**Insight:**
The fog architecture ensures deterministic and low-latency response, making it suitable for real-time cyber-physical systems where delayed decisions can lead to safety risks.

## 6.2 Bandwidth Utilization and Data Reduction



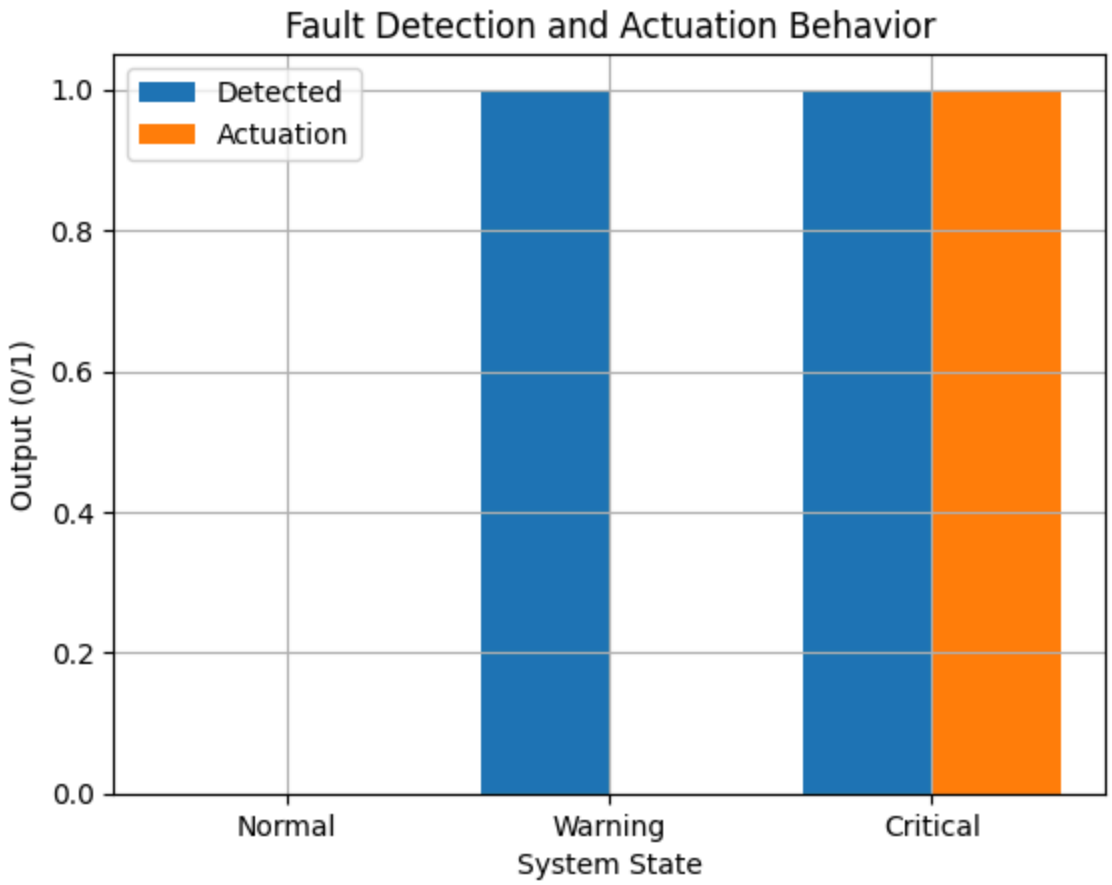

**Comparison of raw telemetry data and processed health vector transmission.**

The system significantly reduces bandwidth consumption by processing raw sensor data locally at the fog node and transmitting only compact **health vectors** to the cloud. As shown in the graph, raw telemetry streams can reach approximately **500 KB/s**, while processed health vectors require only **5–10 KB/s**.

This transformation is achieved through the fog-layer data pipeline, where raw sensor inputs are converted into structured health metrics before transmission .

**Insight:**
The proposed system achieves approximately **90–95% reduction in data transmission**, improving scalability and reducing network dependency while maintaining essential diagnostic information.

## 6.3 Fault Detection and Autonomous Actuation




**System behavior for normal, warning, and critical conditions.**

The fault detection mechanism operates at the fog layer using deterministic rule-based logic. The system classifies vehicle states into Normal, Warning, and Critical conditions based on sensor-derived health parameters.

**For example, a critical condition is triggered when:**

- **Brake temperature exceeds 180°C**
- **Temperature rise rate exceeds 3°C/s**
- **Brake health index falls below 0.4**

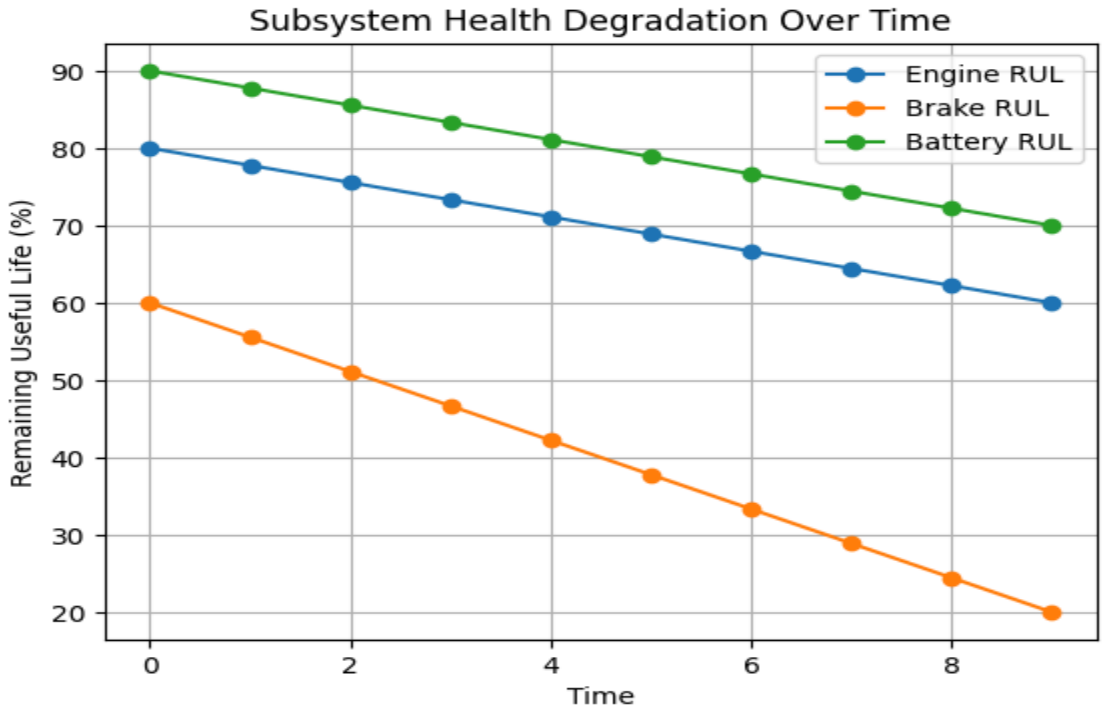
**These conditions are evaluated in real time to determine whether actuation is required**

| Condition | Detection | Actuation |
|---|---|---|

| | | |
|---|---|---|
| Normal | No | No |
| Warning | Yes | No |
| Critical | Yes | Yes |

**Insight:** The system ensures immediate and explainable safety decisions, where actuation is triggered only in critical scenarios, preventing unnecessary interventions.

## 6.4 Vehicle Health Score and Degradation Analysis




The subsystem health degradation graph illustrates the variation of Remaining Useful Life (RUL) for multiple vehicle components, including the engine, braking system, and battery, over time. Each subsystem is represented as an independent trend within the same graph, enabling comparative analysis.

The engine RUL shows a gradual and stable decline, indicating normal wear and tear under operational conditions. In contrast, the brake RUL decreases more rapidly, reflecting higher

stress due to thermal loading and frequent braking events. The battery RUL demonstrates moderate degradation, indicating relatively stable electrical performance.

These subsystem-level metrics are computed at the fog layer and transmitted as part of the processed health dataset

**Insight:**
This visualization highlights that different subsystems degrade at different rates. By presenting all subsystem trends in a single graph, the system enables **comparative analysis and prioritization of maintenance actions**, rather than relying on a single aggregated health score.

## 6.5 Case Study: Brake Overheating Scenario

A real-time fault scenario was simulated using the system dataset. The following parameters were observed:

| Parameter | Value |
|---|---|
| Brake Temperature | 185.6°C |
| Temperature Rise Rate | 4.6°C/s |
| Brake Health Index | 0.39 |
| Decision | Critical |
| Action | Speed Limitation + Cooling Activation |

Based on these inputs, the fog node classified the condition as **critical** and triggered immediate actuation, including limiting vehicle speed and activating the cooling mechanism .

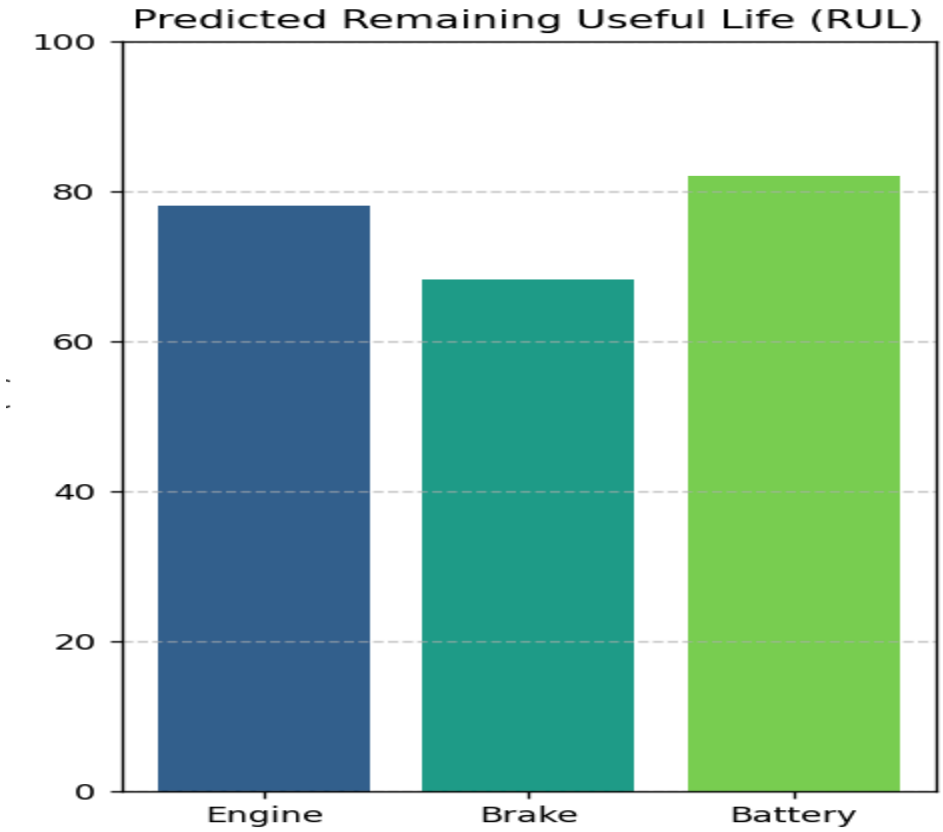
This demonstrates the closed-loop operation of the system:

**Sense → Analyze → Decide → Act**

**Key Insight:**
The system successfully performs **autonomous safety response without cloud dependency**, ensuring reliability even in network-disconnected environments.

# 6.6 AI-Based Predictive Analysis

## 6.6.1 Remaining Useful Life Prediction



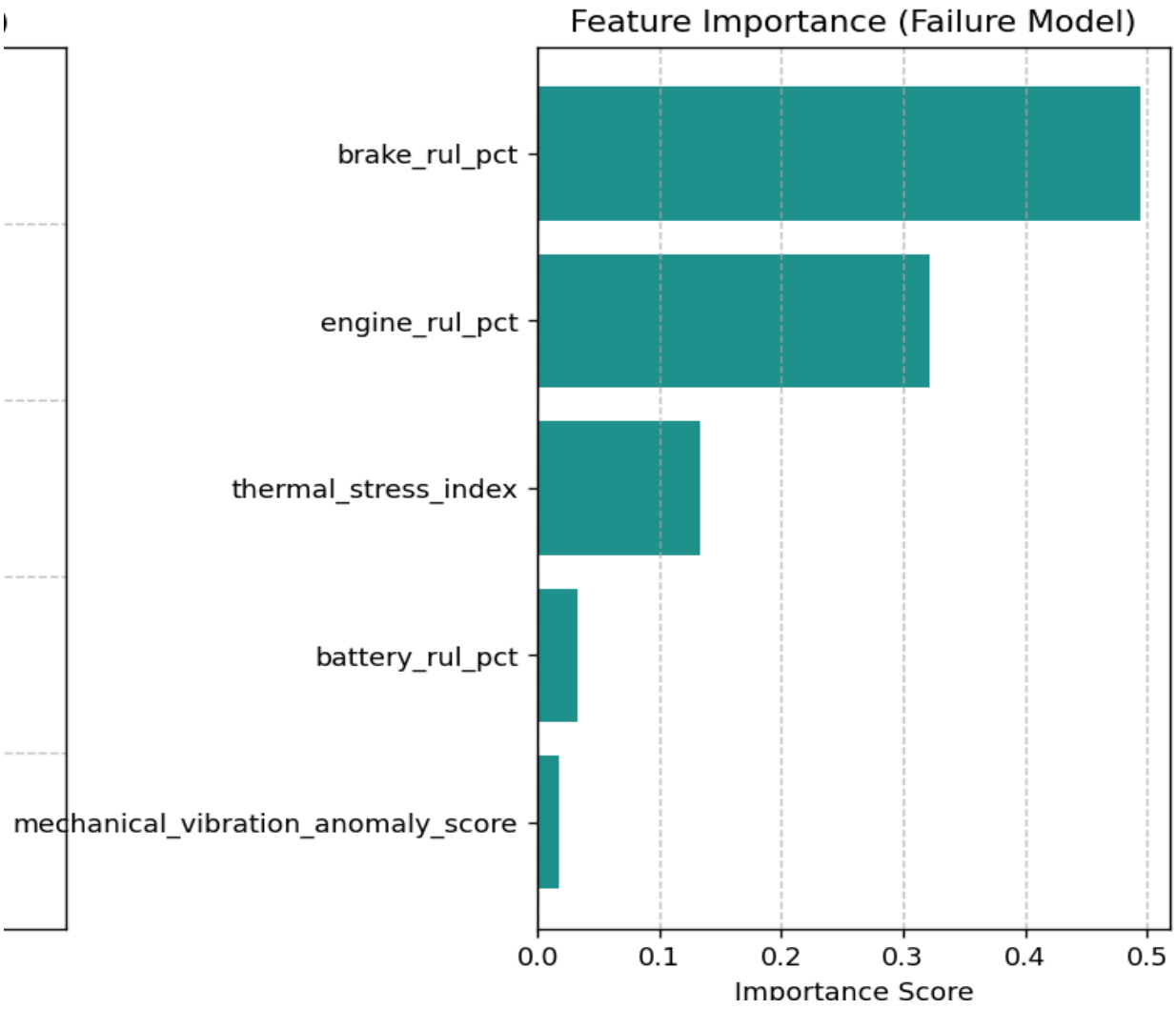

**Predicted Remaining Useful Life using Gradient Boosting Regressor.**

The Gradient Boosting Regressor model is used to estimate the Remaining Useful Life (RUL) of key vehicle components, including the engine, braking system, and battery. The model processes fog-generated health features and outputs component-wise degradation estimates, enabling predictive maintenance.

### 6.6.2 Failure Probability Estimation

The Random Forest Classifier predicts the probability of component failure within a short-term horizon (7 days). This probabilistic output enables proactive maintenance planning and risk assessment.

### 6.6.3 Feature Importance Analysis




**Feature importance derived from Random Forest model.**

The feature importance analysis identifies the most influential parameters contributing to failure prediction. Thermal stress and brake health are observed to have higher importance, indicating their strong role in determining system risk.

**The AI models operate exclusively in the cloud layer and are used for predictive analytics and advisory outputs. They do not have authority over real-time actuation, which remains strictly controlled by the fog layer.**

## 6.6 Overall Discussion

The experimental evaluation demonstrates the effectiveness of the proposed Fog-Based Vehicular Monitoring and Autonomous Fault Response System in addressing the limitations of traditional cloud-centric vehicle monitoring architectures.

The results highlight several key performance improvements:

- Significant latency reduction achieved through fog-level decision-making, enabling real-time response within strict time constraints.
- Efficient bandwidth utilization by transmitting compact health vectors instead of continuous raw telemetry streams.
- Reliable fault detection and autonomous actuation using deterministic rule-based logic at the fog layer, ensuring immediate safety intervention.
- Enhanced predictive maintenance capability through AI-driven models, including Gradient Boosting for Remaining Useful Life (RUL) estimation and Random Forest for failure probability prediction.

Unlike conventional systems that rely heavily on cloud infrastructure, the proposed architecture ensures that safety-critical decisions are executed locally at the fog node, making the system resilient to network disruptions and communication delays. The strict separation between fog and cloud layers further improves system reliability and security, as actuation authority is confined exclusively to the fog layer, while the cloud is responsible only for analytics and advisory functions .

Additionally, the integration of AI-based predictive models enables the system to move beyond reactive diagnostics toward proactive maintenance planning, allowing early identification of component degradation and failure risk.

Overall, the proposed system successfully transforms the vehicle into an intelligent cyber-physical system capable of real-time monitoring, autonomous fault response, and data-driven predictive analytics. By combining fog computing with machine learning, the architecture effectively overcomes challenges related to latency, bandwidth consumption, and lack of autonomous decision-making in existing vehicular monitoring solutions.

# Comparative Analysis

The following analysis compares the performance, working methodology, and results of the proposed Fog-Adaptive Vehicular Monitoring & Control System against existing state-of-the-art research and patented technologies identified in the literature.

## 1. Performance and Latency

Existing cloud-centric systems, such as those discussed by Zhang et al. (IEEE, 2023), face significant challenges regarding communication latency and network dependency. In contrast, our system utilizes a **Mobile Phone-based Fog Node**.

| Metric | Existing Cloud-Centric Systems | Proposed Fog-Adaptive System |
|---|---|---|
| Response Time | High (subject to network latency) | Millisecond-level (local processing) |
| Connectivity | Requires continuous internet | Offline-capable "Survival Mode" |
| Bandwidth | High (streams raw telemetry) | Low (transmits compressed health vectors) |

## 2. Working Methodology: Sensing and Data Processing

Traditional systems often rely on a fragmented sensing architecture, requiring numerous physical sensors for each subsystem.

- **Existing Methodology**: Focuses on single-modality analysis, such as Shah et al. (2025) who looked exclusively at vibration. This limits the ability to assess holistic vehicle health.
- **Proposed Methodology**: Implements **Non-Intrusive Vehicular Load Monitoring (NIVLM)**. By analyzing power-line harmonics via an ESP32 and CT sensors , the system achieves **AI Fault Disaggregation**. This allows for component-level identification with approximately **85% fewer physical sensors**.
- **Data Transformation**: Raw signals (Electrical, Thermal, Vibration) are processed locally into **Health Vectors**. For example, the **Thermal Stress Index** is calculated by clamping normalized temperature and rise rates

**thermal_stress_index = clamp(0.7 * normalized_temp + 0.3 * normalized_rise, 0, 1)**

## 3. Autonomy and Actuation Results

A critical gap in existing patents (e.g., **US 20240112584 A1**) is the lack of closed-loop actuation; they provide diagnostics but no real-time intervention.

- **Existing Results**: Primarily "Monitoring-Only". Faults are identified, but safety depends on driver or cloud-initiated response.
- **Proposed Results**: Enables **Autonomous Actuation**. The system executes safety-critical logic locally. For instance, **Thermal Protection** is automatically activated if:

  **thermal_protection_active = (brake_temp_c > 180) AND (brake_temp_rise_rate_c_per_sec > 3.0) AND (brake_health_index < 0.4)**

This triggers immediate hardware responses, such as limiting vehicle speed to **40 km/h** or enabling cooling fans.

## 4. Comparison Summary Table

| Feature | Literature/Patents | Proposed FAVCL System |
|---|---|---|
| Decision Origin | Predominantly Cloud | Fog Layer (Smartphone) |
| Hardware Overhead | High (Multi-sensor arrays) | Minimal (CT sensors/NIVLM) |

| | | |
|---|---|---|
| Safety Feedback | Delayed/Advisory only | Real-time Actuation (Relay control) |
| User Interface | Static diagnostics | Interactive Digital Twin |

# Tools and Technologies Used

The system is implemented using a layered architecture consisting of hardware sensing, mobile fog computing, cloud AI processing, backend services, and frontend visualization. Each layer uses specific tools and technologies suited to its role.

### Hardware Requirements

- ESP32 microcontroller for sensor data acquisition and actuation control
- Thermal sensors for engine, brake, and radiator temperature monitoring
- Electrical sensors for battery voltage and current measurement
- Motion and vibration sensors (IMU / gyroscope) for mechanical analysis
- Relay modules and GPIO-controlled actuators for safety actions
- Communication interfaces such as I2C, SPI, ADC, and GPIO
- Android smartphone of the vehicle user used as the fog/edge computing node inside the vehicle

### Software Requirements

- Embedded Layer
    - C / C++ using Arduino framework or ESP-IDF
    - ESP32 sensor interfacing libraries

- Communication Layer

    - HTTP-based REST communication (currently implemented)
    - MQTT protocol (planned/alternative communication method)
    - Eclipse Mosquitto broker (for MQTT-based deployment)
    - JSON data format for telemetry packets

- Fog Layer (Mobile Phone)
    - Android platform
    - Python service layer and/or Kotlin/Java
    - NumPy and SciPy for signal processing
    - Pandas for health vector construction
    - HTTP client for communication
    - TensorFlow Lite for optional AI inference

- AI / Machine Learning Layer
    - Scikit-learn for predictive modeling
    - TensorFlow / TensorFlow Lite for deep learning models
    - Regression models for Remaining Useful Life estimation
    - Anomaly detection classifiers

- Backend Layer
    - FastAPI framework (Python)
    - MongoDB NoSQL database
    - REST-based API architecture
    - PyMongo for database interaction

- Frontend Layer
    - React.js with TanStack routing
    - Tailwind CSS for styling
    - ShadCN UI and chart Blocks React component library
    - Three.js for digital twin visualization
    - Firebase for authentication

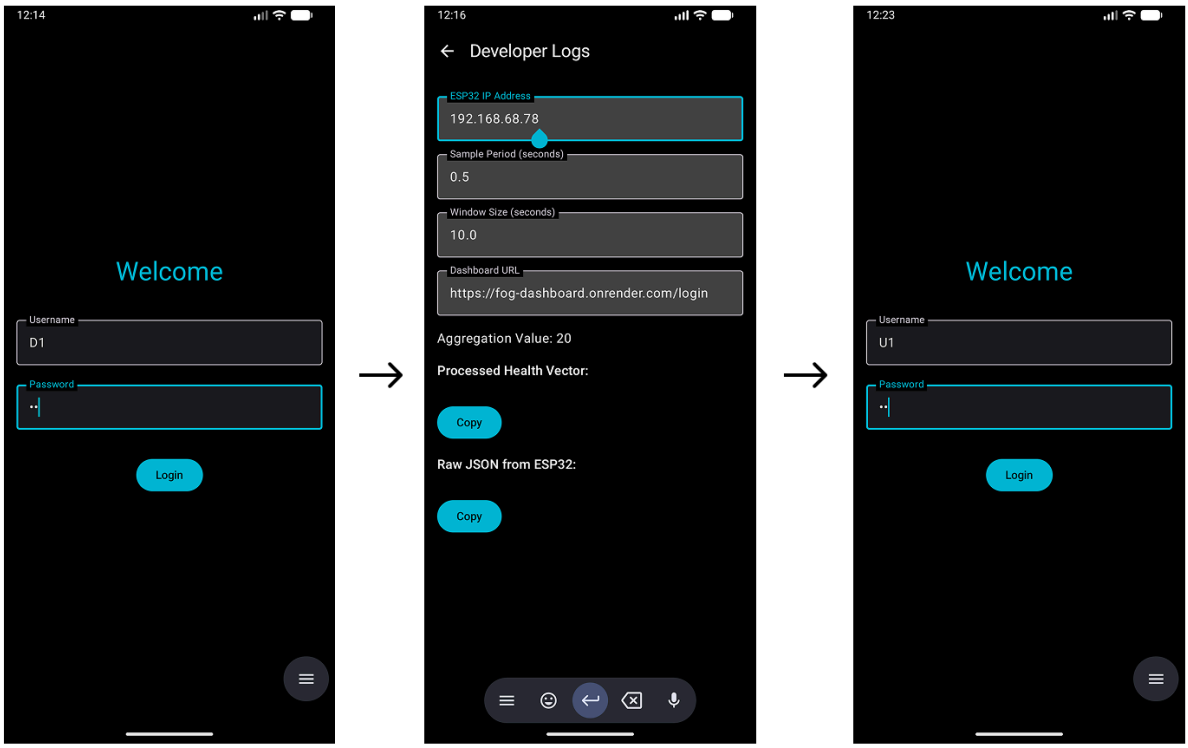
    - React Query (part of Tanstack) for data fetching and caching

- Deployment and DevOps
    - Git and GitHub for version control
    - Render for backend and frontend deployment
    - Environment variable configuration for runtime setup
    - curl for API testing
    - pytest for automated endpoint validation
    - requirements.txt and package.json for dependency management

# User Manual.

## Step 1 : Installation and Initial Setup by Dealership.

The mobile application is first installed by the dealership technician. The application runs a background service(acting as the edge layer) responsible for collecting sensor data from the vehicle's ESP32-based central node. The technician configures the IP address of the ESP32 and verifies that sensor data is being received correctly before leaving the application running in the background.



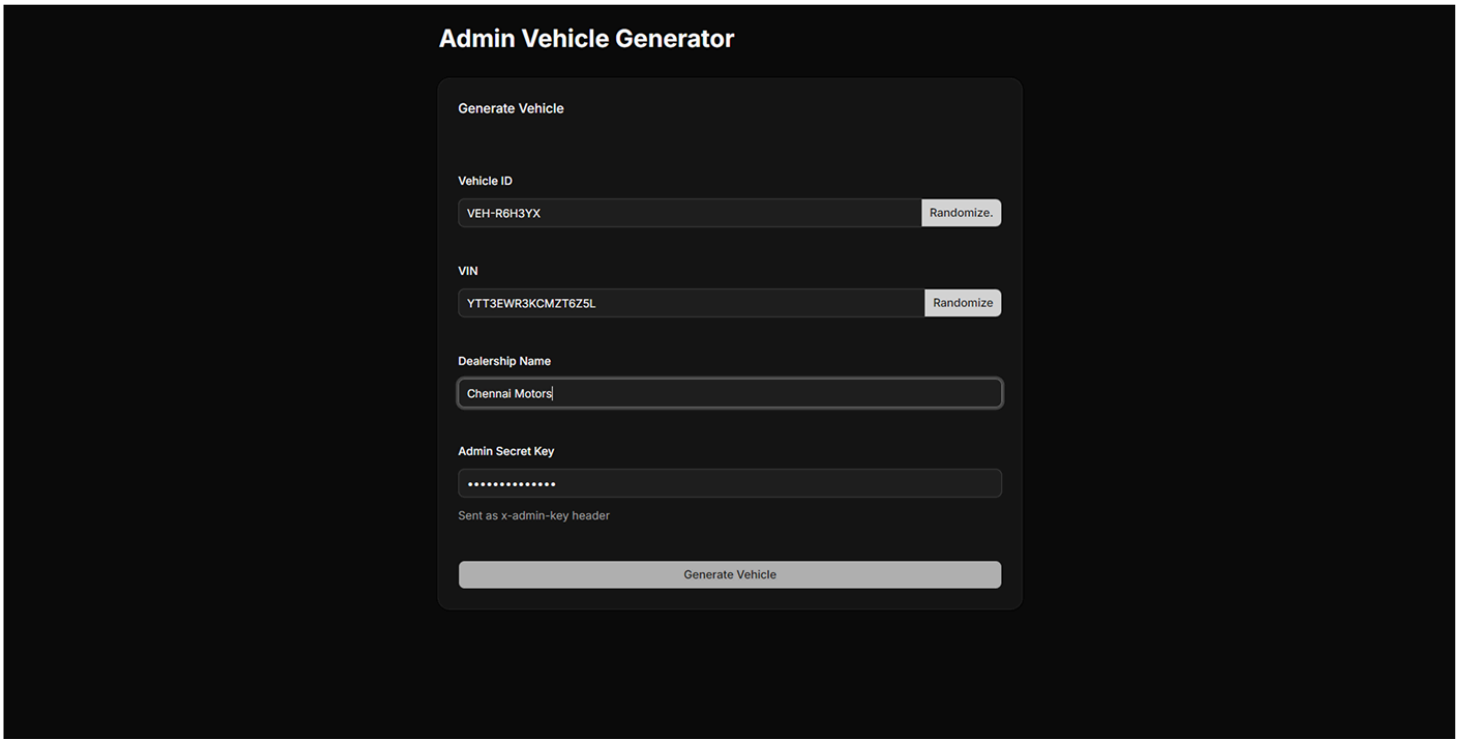

1.1 Dealership enters the admin credentials

1.2 Dealership configures the IP address of the sensor node, polling frequency and the frontend URL user will use to see vehcile data.

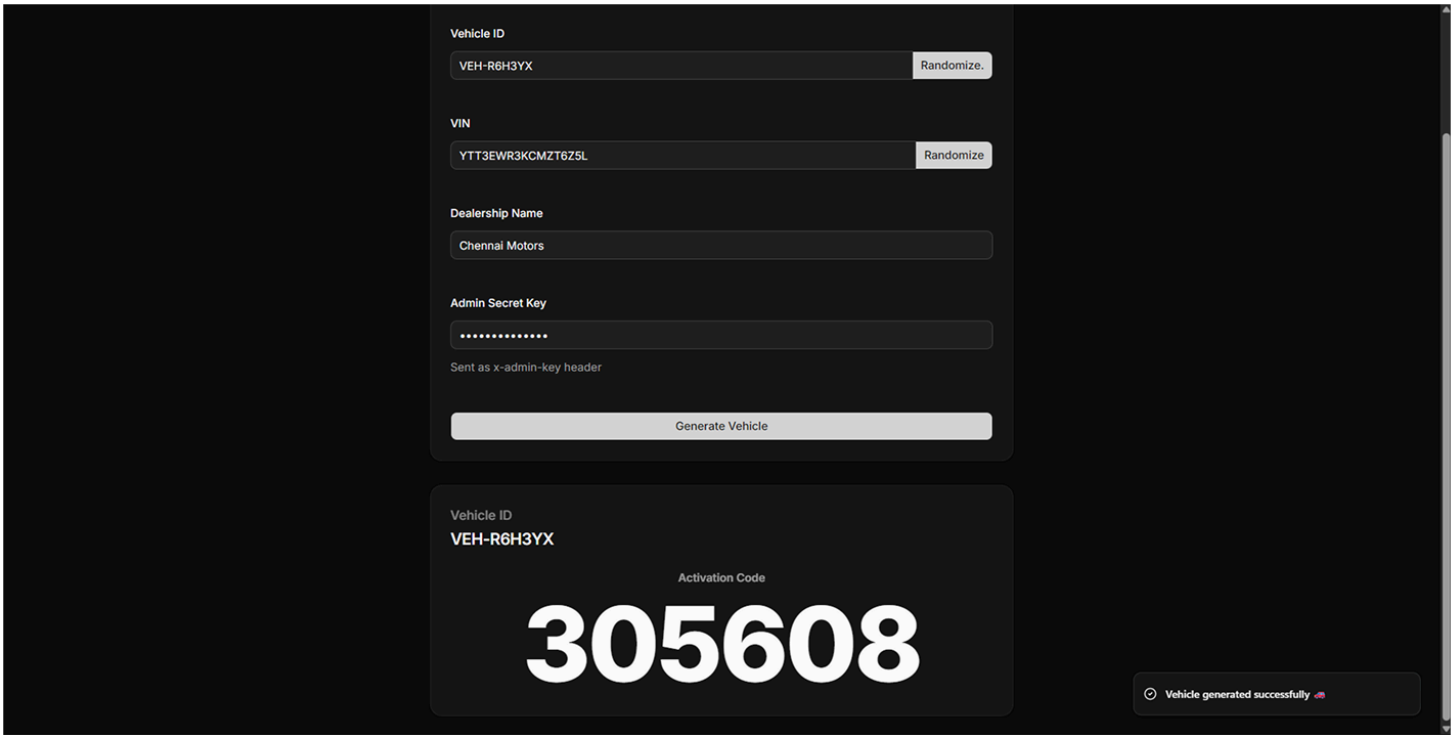
1.3 Now user can log in to see the card dashboard using the credentials provided by the dealership.

## Step 2 : Vehicle Record Creation by Dealership.

The dealership creates a vehicle record using the administrative interface. A unique vehicle ID and activation code are generated and shared with the customer for claiming the vehicle.



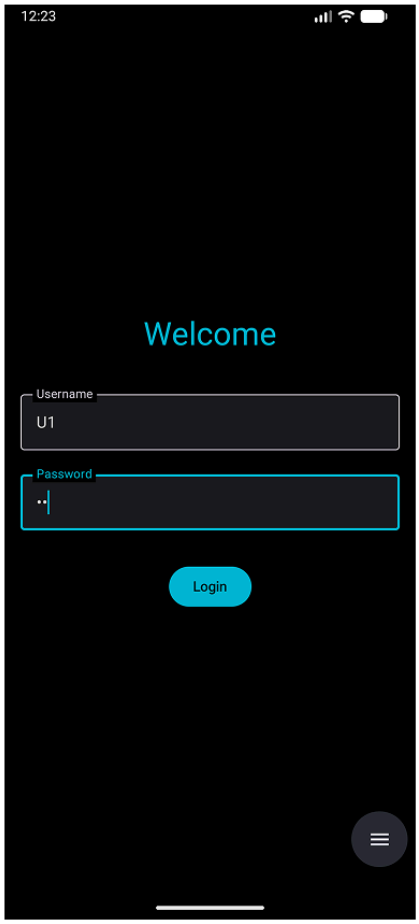

2.1 : Dealership Employee uses the admin page of the dashboard to generate the **VehicleID**
Dealership needs to give the **admin-key** for authentication and also needs to give a valid 16 Digit Vehicle Identification Number.



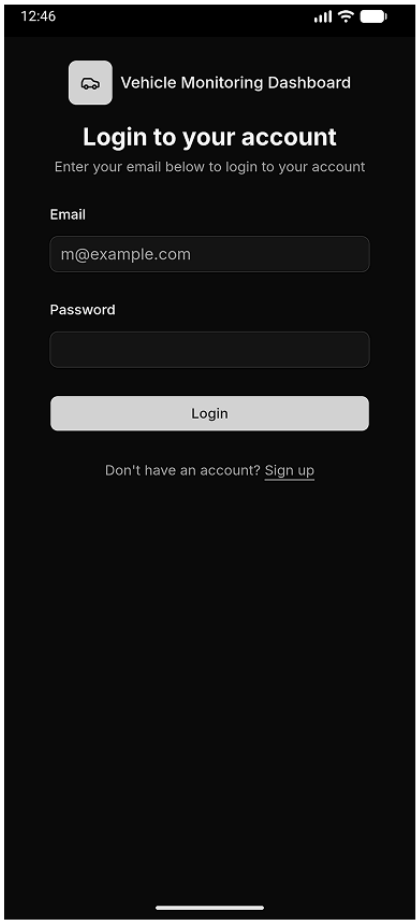

2.2 : An activation code gets generated which is shared to the customer along with the vehicleID to claim the vehicle ownership.

## Step 3 : User Signup.

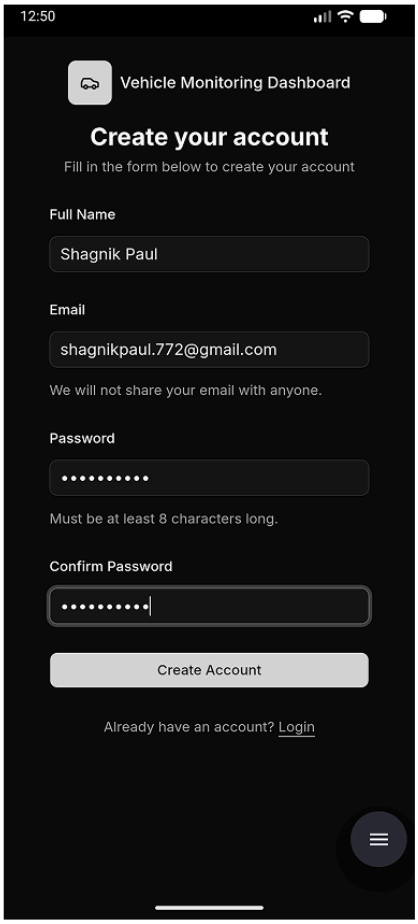
The customer creates an account using the signup page of the dashboard application. Authentication is handled securely using Firebase.




3.1 logs in to see the card dashboard using the credentials provided by the dealership.



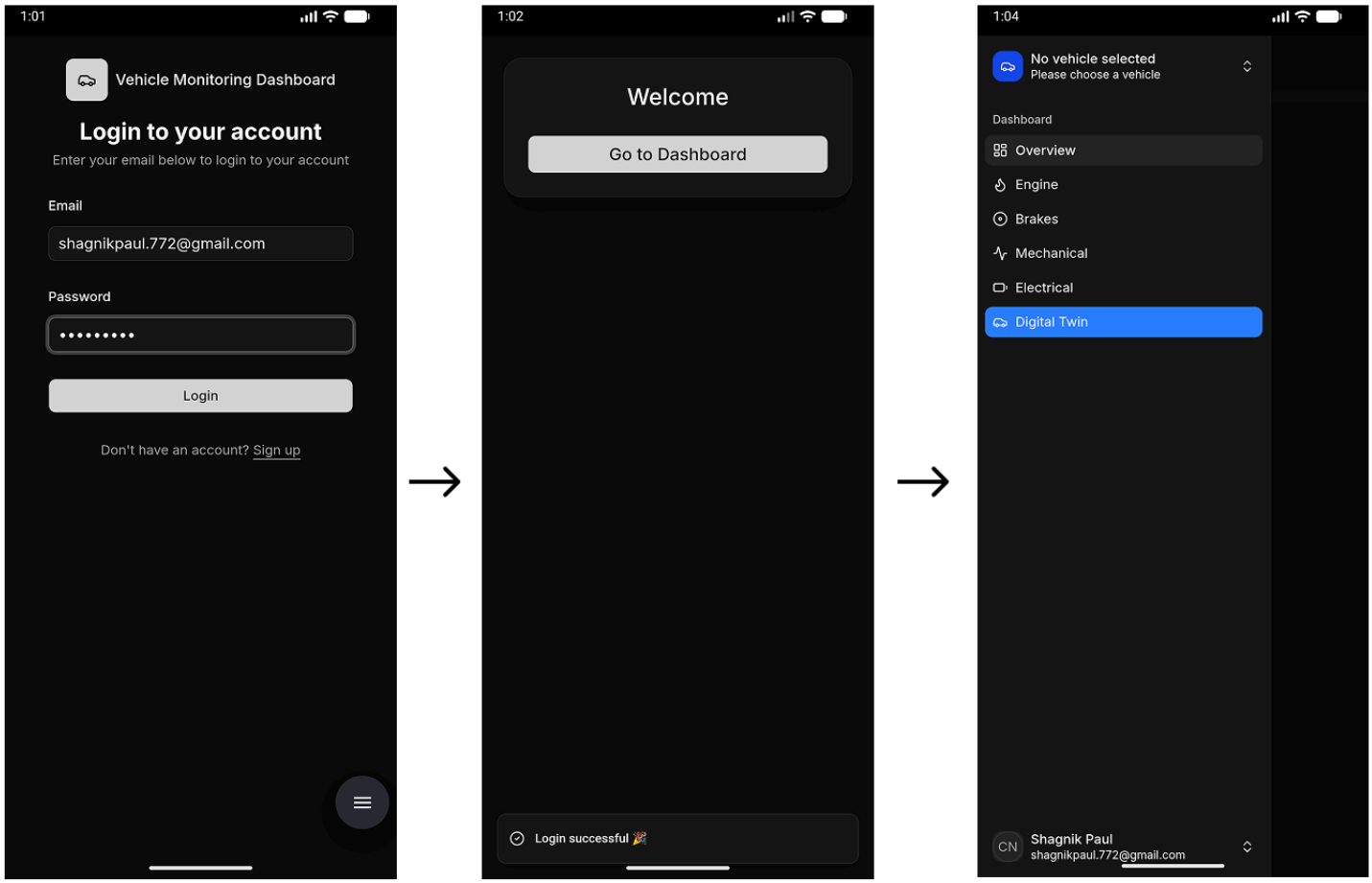

3.2 User is presented with the log-in screen of the web-dashboard. User needs to create an account first for first time usage using the **SignUp** button below the form.




3.3 User enters the sign up details and clicks on the Create Account button.

## Step 4 : User LogIn.

After successful registration, the user logs into the system. Once authenticated, protected dashboard routes become accessible.



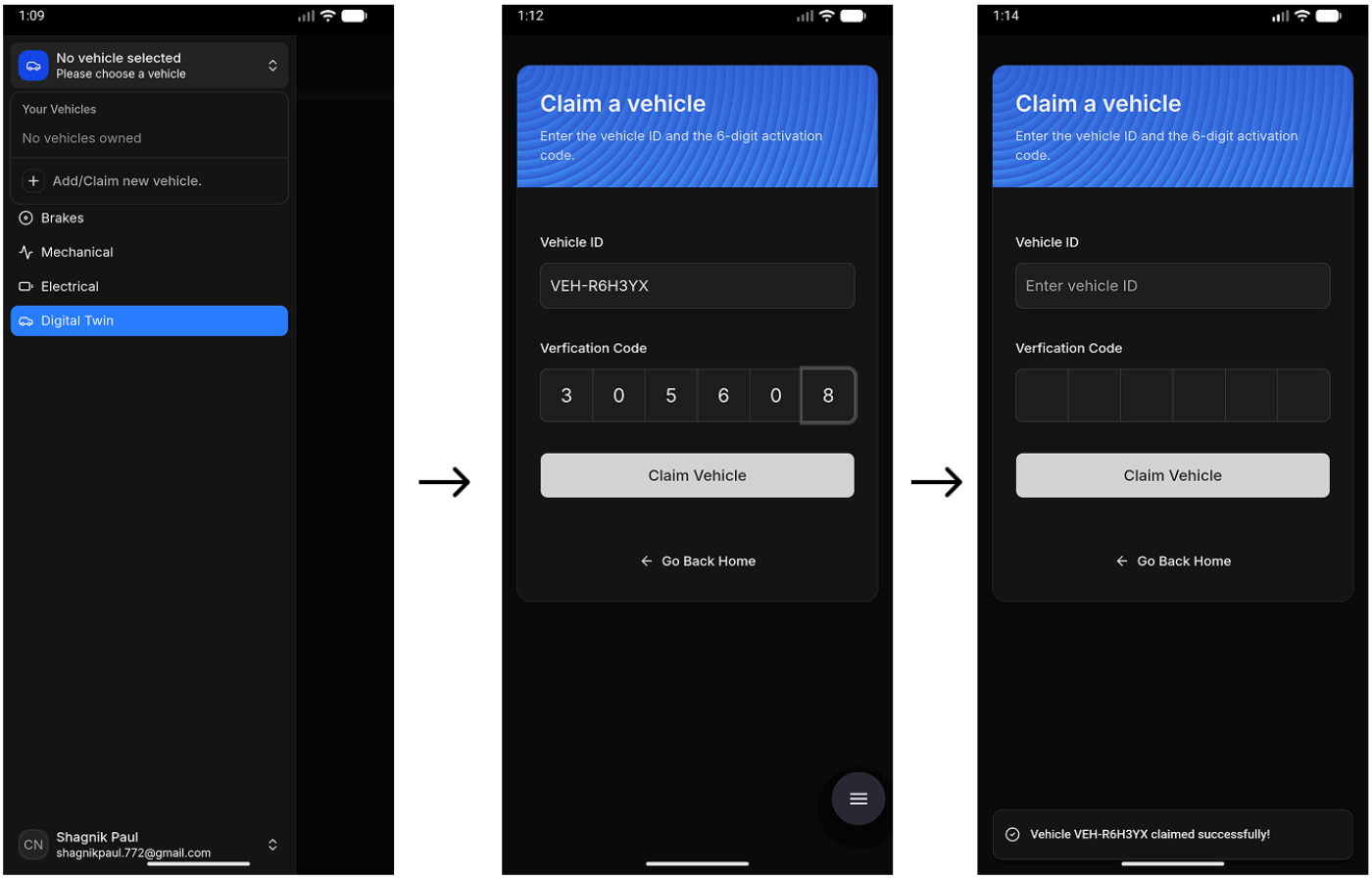

4.1 User enters their account credentials then clicks on Login

4.2 If the credentials were correct user is redirected to the homepage. Click on dash board button to go to dashboard page.

4.3 Dashboard page with no vehicle selected yet.

## Step 5 : Claim vehicle ownership by the user.

After login, the dashboard initially shows no vehicles assigned to the user. The user must claim a vehicle using the activation code provided by the dealership. The user navigates to the "Claim Vehicle" page from the sidebar and enters the vehicle ID and activation code provided by the dealership. Upon successful validation, the vehicle is added to the user’s account and becomes available in the vehicle selection dropdown.



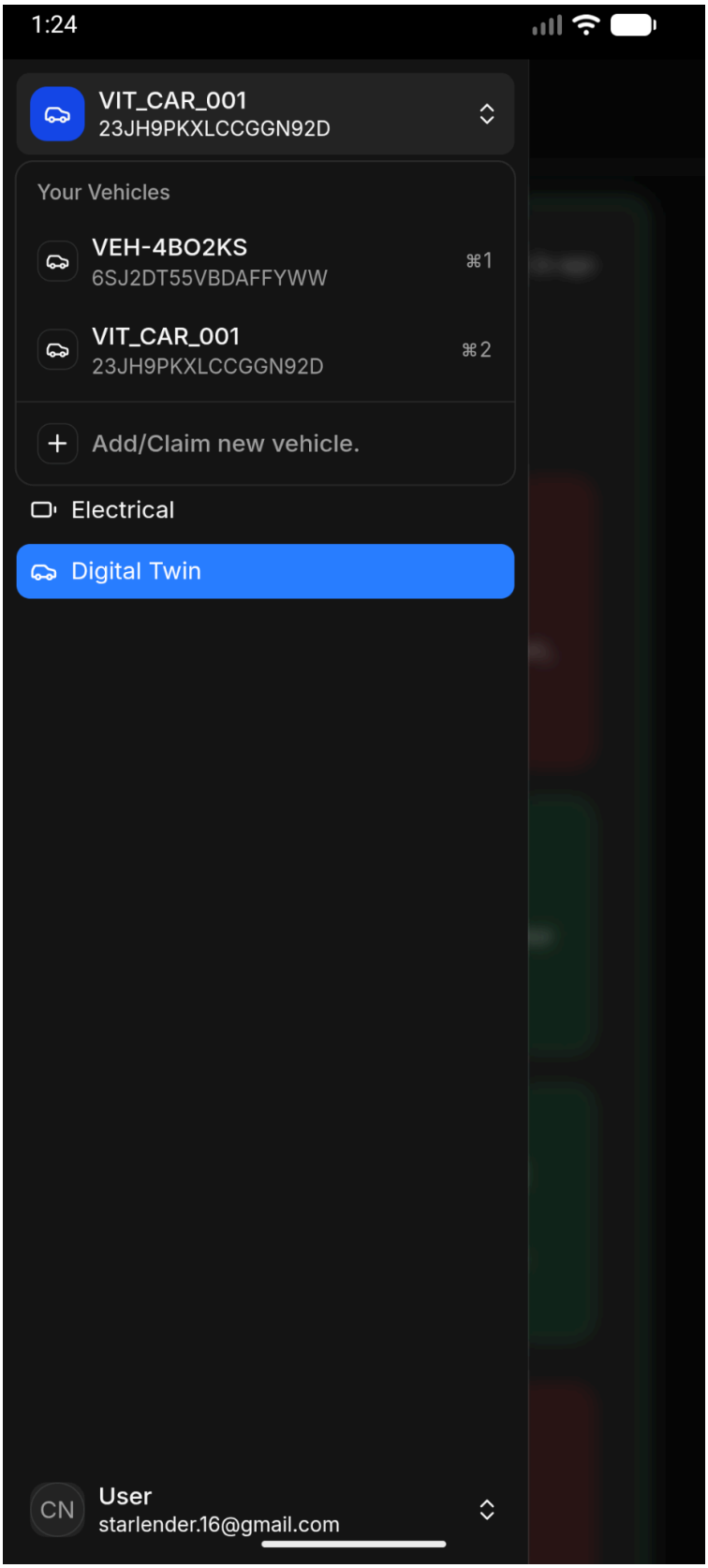

5.1 User Opens the sidebar and clicks the Claim Vehicle button from the vehicle selection dropdown menu.

5.2 User enters the Activation code and the vehicleID received from the dealership and then click the claim button.

5.3 Success message shown upon verification of the details and claim transfer ship.

## Step 6: Choose a vehicle and Navigation through different dashboard screens.

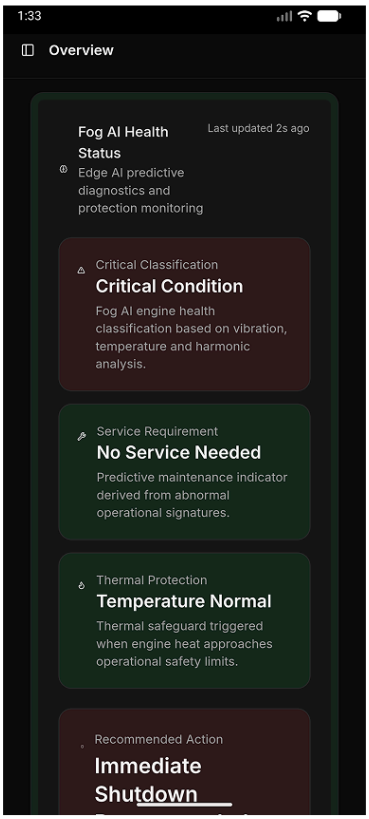
Upon successful validation, the vehicle is added to the user's account and becomes available in the vehicle selection dropdown.User can now navigate through different screens using the SideBar navigation menu.



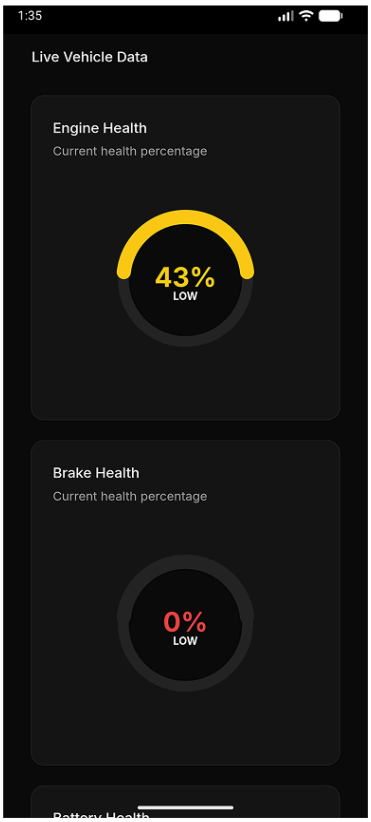

**6.1 -** Choose a Vehicle from the drop down selection of vehicles. If none are shown add a new vehicle using the **Claim Vehicle Button** then provide the activation code and the vehicle id given by the dealership to claim the ownership.

Major Screens in the dashboard

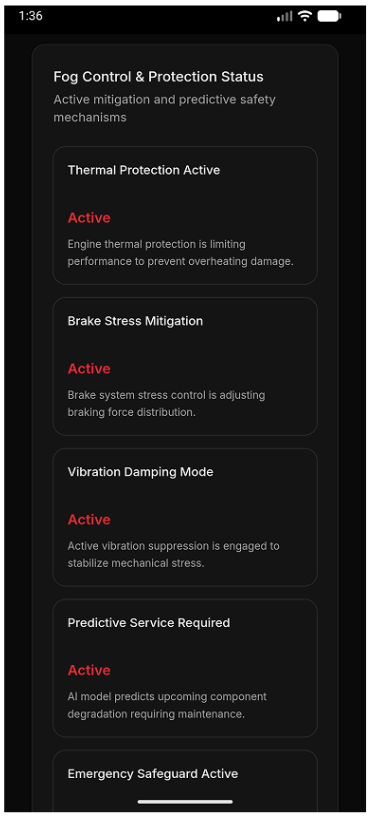
- **Main Dashboard View** - The main dashboard displays AI-generated recommendations, system health gauges, and actuation status indicators.




AI Recommendation Details

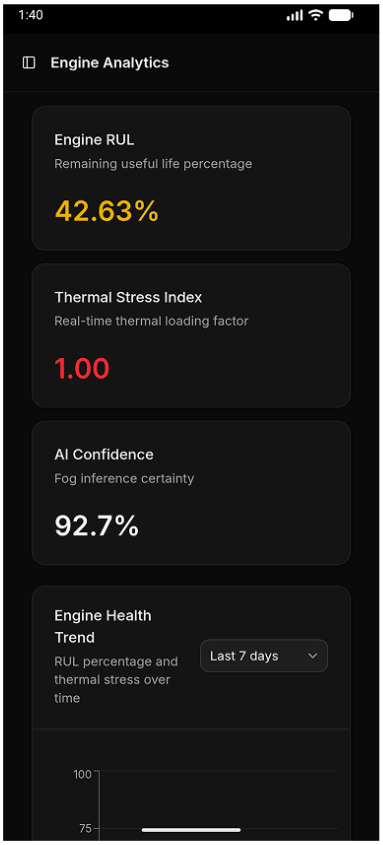



Live Health data of different vehicle Components in Gauge/Meter style



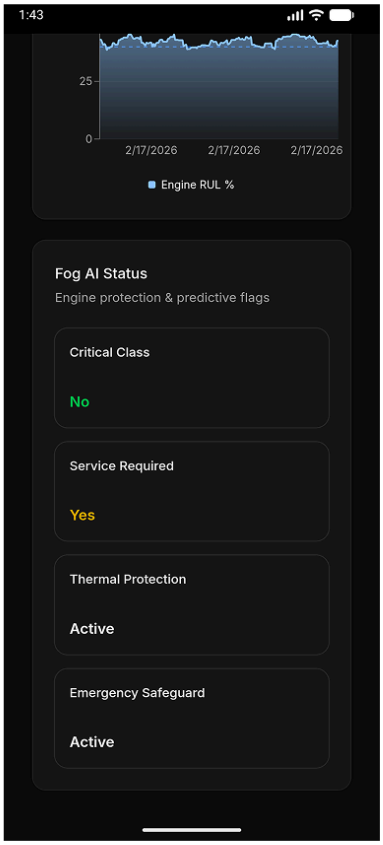

Actions taken by the edge layer as a preventive / emergency procedure if any.

- **Engine Health Dashboard** - The engine dashboard provides Remaining Useful Life (RUL), thermal stress index, and real-time chart visualization.



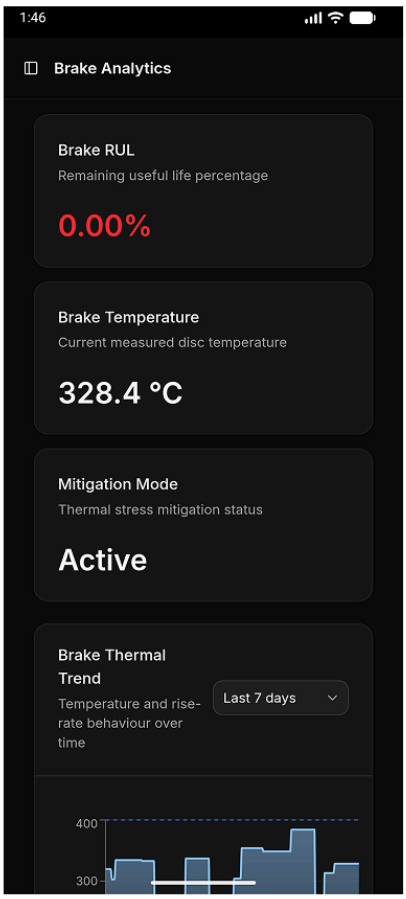

Engine RUL and thermal Stress info


Engine RUL Last 100 records chart visualization

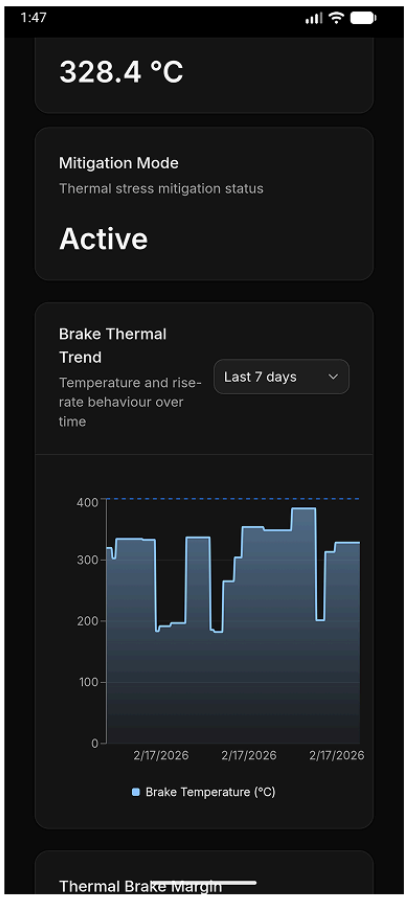



Actions taken by the edge layer particularly on the engine as a preventive / emergency procedure if any. Also maintenance recommendations are shown

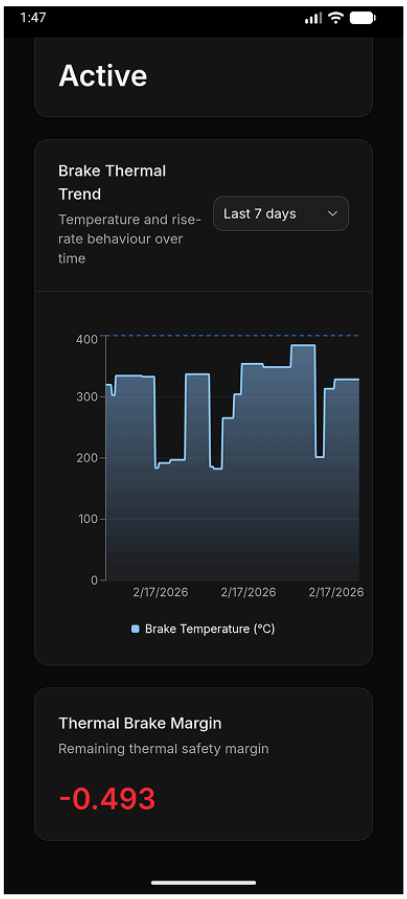
- **Brake Health Dashboard -** The brake dashboard displays braking system health metrics and trend charts.




Brake RUL and thermal info

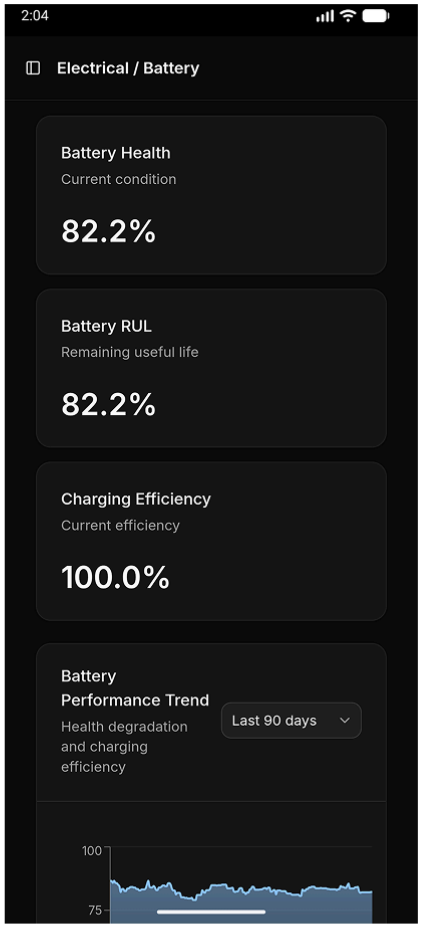



Brake Temperature Last 100 records chart visualization

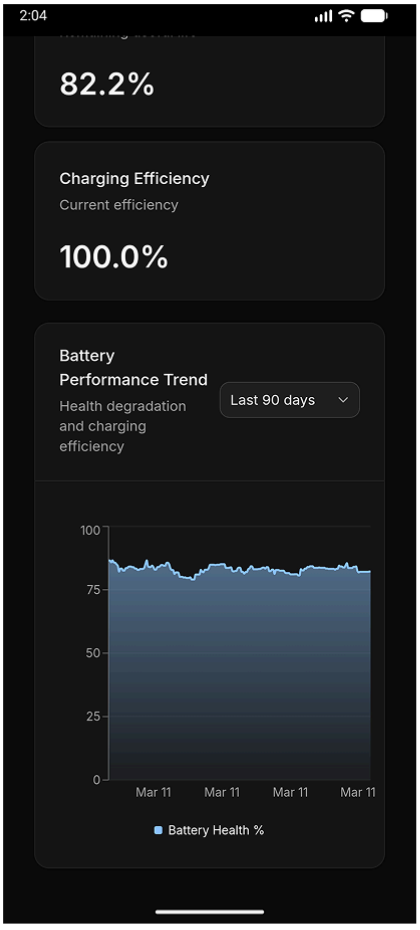



Thermal Brake Margin metric

- **Electrical Health Dashboard** - Electrical subsystem metrics such as battery health and charging efficiency are shown in this dashboard.

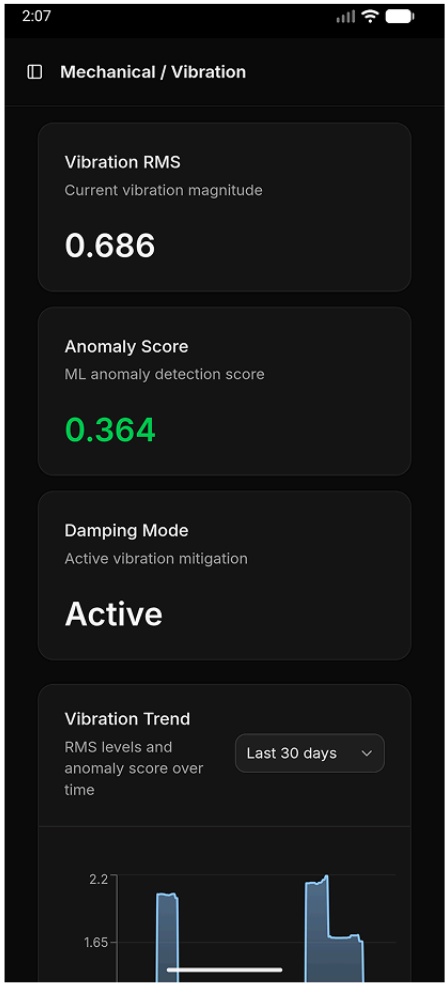



Battery Health Metrics

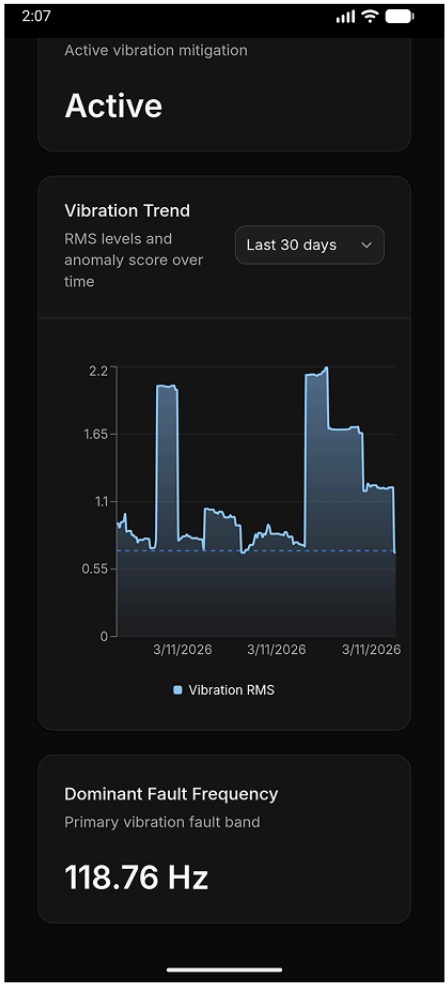



Battery Health Last 100 records chart visualization

- **Mechanical Health Dashboard** - Mechanical vibration metrics and anomaly detection results are displayed.

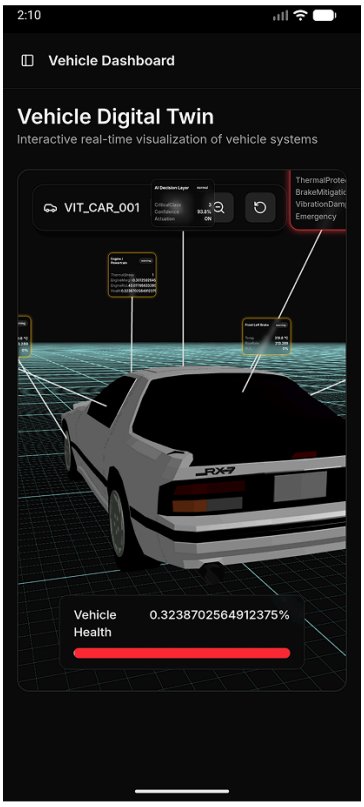



Mechanical Wear Metrics

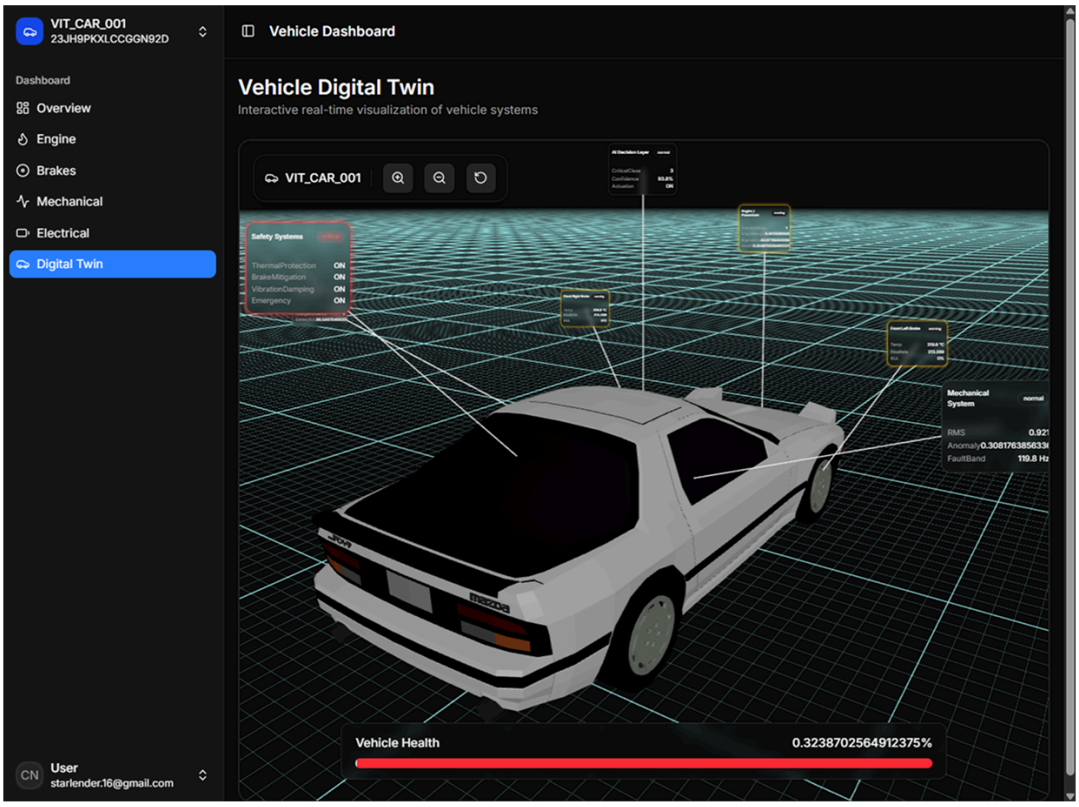



Last 100 records vibration amount trend and dominant frequency of it

- **Digital Twin Visualization** - The digital twin view provides a 3D interactive vehicle model with component-level health overlays.




Mobile View




Desktop View

# References


- [1] Kapoor, Dimple, Deepali Gupta, and Mudita Uppal. “Analyzing the impact of edge, fog and cloud computing on predictive maintenance in the Industrial Internet of Things.” *Discover Computing*, vol. 28, 2025.
- [2] Teoh, Y. K., S. S. Gill, and A. K. Parlikad. “IoT and Fog-Computing-Based Predictive Maintenance Model for Effective Asset Management in Industry 4.0 Using Machine Learning.” *IEEE Internet of Things Journal*, vol. 10, no. 3, 2021, pp. 2087–2094.
- [3] Ilyas, Abeera, et al. “Software architecture for pervasive critical health monitoring systems using fog computing.” *Journal of Cloud Computing*, vol. 11, 2022.
- [4] Zero, Enrico, Mohamed Sallak, and Roberto Sacile. “Predictive Maintenance in IoT-Monitored Systems for Fault Prevention.” *Journal of Sensor and Actuator Networks*, vol. 13, no. 5, 2024.
- [5] Dhall, Rohit, and Vijender Kumar Solanki. “An IoT Based Predictive Connected Car Maintenance Approach.” *International Journal of Interactive Multimedia and Artificial Intelligence*, 2017.
- [6] Goethals, Tom, Filip De Turck, and Bruno Volckaert. “Near real-time optimization of fog service placement for responsive edge computing.” *Journal of Cloud Computing*, vol. 9, 2020.
- [7] Thakkar, Hiren Kumar, et al. *Predictive Analytics in Cloud, Fog, and Edge Computing: Perspectives and Practices of Blockchain, IoT, and 5G.* Springer, 2023.
- [8] “IoT-based predictive maintenance for fleet management.” *Procedia Computer Science*, vol. 151, 2019, pp. 607–613.
- [9] Kerner, Sean Michael. “IoT and digital twins: How they work together, with examples.” *TechTarget*, 2025, https://www.techtarget.com/iotagenda/feature/IoT-and-digital-twins-How-they-work-together-with-examples.
  A digital twin is described as a “virtual representation of a real-world entity” continuously updated using IoT data streams.

- [10] Butt, Sahar. “How to Build a Digital Twin? Definition, Process & More.” *VentureDive*, 2023, https://www.venturedive.com/insights/blogs/how-to-build-digital-twins.
- [11] “FastAPI Documentation.” FastAPI, https://fastapi.tiangolo.com/
- [12] “MongoDB Atlas Documentation.” MongoDB, https://www.mongodb.com/docs/
- [13] “React Documentation.” React, https://react.dev/
- [14] “Firebase Authentication.” Google Firebase, https://firebase.google.com/docs/auth
- [15] “Three.js Documentation.” Three.js, https://threejs.org/docs/
- [16] “MQTT Protocol Specification.” MQTT, https://mqtt.org/
- [17] “ESP32 Technical Reference Manual.” Espressif Systems, https://www.espressif.com

# Appendix

- Link to the presentation : LINK
- Link to the video presentation: LINK
- Github Link containing the source code files and documentation : GITHUB LINK
- Steps to execute the project can be found here in case the link is not working please visit the Docs folder of our main GitHub repo to read the documentation there.